CSE752: Deep Neural Networks

Contents

Introduction to Neural Network **2**

Week 1: The Neuron and the MLP **5**

Introduction to Neural Network

Computation of a Single Neuron: $f(w^\top x + b)$

A neuron computes $f(w^\top x + b)$, because the input is usually a vector.

- **Single input (scalar):** $a = f(wx + b)$
- **Multiple inputs (vector $x \in \mathbb{R}^d$):** $z = w^\top x + b$, then $a = f(z)$

The neuron does two things: an affine step (weighted sum plus bias) that gives the pre-activation z, then a nonlinearity f applied to that scalar. The nonlinearity is the only place a curve can enter. If f were the identity, stacking layers would collapse into one affine map.

For a whole layer, the same idea is written with a matrix: $z = Wa^{(l-1)} + b$, $a = f(z)$, with f applied elementwise.

Number of Inputs to a Neuron

A neuron can take multiple values as input. A single input is just the special case $d = 1$.

With d inputs, the neuron has **one weight per input** but only **one bias**: $x \in \mathbb{R}^d$, $w \in \mathbb{R}^d$, $b \in \mathbb{R}$. It multiplies each input by its own weight, sums the results, adds the bias, and applies f:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = w^\top x + b, \qquad a = f(z)$$

Whatever the number of inputs, the output is always **one scalar**.

For example, in a $784 \to 128 \to 64 \to 10$ network, each neuron in the first hidden layer takes all 784 inputs, so it has 784 weights and 1 bias. This is why a layer's parameter count is $n_{\text{out}} \cdot n_{\text{in}} + n_{\text{out}}$: n_{out} neurons, each with n_{in} weights and one bias.

What Is a Layer?

A layer is a group of neurons that all receive the same input vector and compute in parallel, each producing one scalar. Stacking those scalars gives the layer's output vector, which becomes the input to the next layer.

If a layer has n_{out} neurons and receives n_{in} inputs, then every neuron has its own n_{in} weights and its own bias. Stacking the weight vectors as rows gives one matrix, so the whole layer computes in a single step:

$$z = Wa_{\text{prev}} + b, \qquad a = f(z)$$

with W of shape $(n_{\text{out}} \times n_{\text{in}})$, and b, z, a of length n_{out}. The activation f is applied to each coordinate independently.

Example: a $2 \to 2 \to 1$ network.

- **Input layer:** just x, the raw inputs. It has no weights and does no computation.
- **Hidden layer:** 2 neurons, each seeing both inputs ($W^{(1)}$ is 2×2). It is called hidden because its outputs are neither the inputs nor the final prediction.
- **Output layer:** 1 neuron, seeing both hidden outputs ($W^{(2)}$ is 1×2), producing $\hat{y}$.

A layer where every neuron sees every input from the previous layer is called a **fully connected** (or dense) layer. A network with several of these in a row is an MLP.

Do Input Layer Neurons Perform Computation?

No. The input layer's "neurons" are just placeholders that hold the raw input values $(x_1, x_2, \dots)$. They have no weights, no bias and no activation, so nothing is computed there.

The first real computation happens in the first hidden layer, which takes the input values and applies $f(Wx+b)$. The weight matrix $W^{(1)}$ is described as connecting the input layer to the first hidden layer, but it belongs to the hidden layer's neurons: its shape is $(n_{\text{hidden}} \times n_{\text{input}})$.

This is also why a $2 \to 2 \to 1$ network is usually called a "2-layer network" (hidden + output). The input layer is drawn as circles for visualization, but it is not counted because it does no computation.

Input Layer Size for a Tabular Dataset

Suppose a tabular dataset has 1000 rows and 4 columns. The number of input neurons depends on what the columns are.

- **All 4 columns are input features (target is separate):** 4 input neurons, one per feature. The number of rows does not matter; each row is passed in as a vector $x \in \mathbb{R}^4$, and the 1000 rows are 1000 training examples.
- **One of the 4 columns is the target (label):** only 3 are features, so the input layer has 3 neurons. The target column is what the output layer is compared against in the loss, not something fed in.
- **Preprocessing can change the input size:** a categorical column with k categories, one-hot encoded, becomes k input neurons instead of 1. Dropping a column, or adding engineered features, changes the count too.

The rule is: **input layer size = number of numeric input values per row after preprocessing**, not the raw column count. The label is excluded.

Since there are 1000 rows, they are typically passed in batches (say 32 at a time). The input is then a matrix of shape $(\text{batch} \times n_{\text{features}})$, but the network's structure is unchanged: batch size never affects the number of neurons or weights.

Output Layer Design Based on the Label Type

For a dataset with 3 numerical features and 1 label, the input layer has 3 neurons. The label is not fed in; it only appears in the loss, where it is compared against the network's output.

The output layer depends on the label type:

- **Regression (continuous label):** 1 output neuron, usually with no activation (or identity).
- **Binary classification:** 1 output neuron with sigmoid.
- **Multi-class classification with k classes:** k output neurons with softmax.

A network such as $3 \to 8 \to 4 \to 1$ would work for regression or binary classification. The hidden sizes (8 and 4) are a free choice; only the input size (3) and the output size (set by the task) are fixed by the data.

One-Hot Encoding of Categorical Features

A neuron computes $w^\top x + b$, so every input has to be a number, and the number's size has to mean something. A categorical column like color $= \{\text{red}, \text{green}, \text{blue}\}$ is just labels. If we encode them as red $= 1$, green $= 2$, blue $= 3$, we tell the network that blue is "three times red" and green sits "between" them. The weight multiplies that number, so the network is forced to treat the categories as points on a line with a fixed order and spacing, which is usually false.

One-hot encoding avoids this by giving each category its own 0/1 column:

color	is_red	is_green	is_blue
red	1	0	0
green	0	1	0
blue	0	0	1

One original column becomes k columns (here $k = 3$), and exactly one of them is 1 in each row. Since the input layer has one neuron per input value, those k columns need k input neurons.

This also fixes the weight problem. The first hidden neuron now has a separate weight for each category: $w_{\text{red}}, w_{\text{green}}, w_{\text{blue}}$. Its pre-activation is

$$z = w_{\text{red}} \cdot \text{is_red} + w_{\text{green}} \cdot \text{is_green} + w_{\text{blue}} \cdot \text{is_blue} + (\text{other terms}) + b$$

For a red row, only w_{red} contributes. For a green row, only w_{green} does. So the network learns an independent effect for each category, with no imposed ordering or spacing.

Example: a dataset has 3 numerical features plus one categorical column with 4 categories. After one-hot encoding, each row is a vector of length $3 + 4 = 7$, so the input layer has 7 neurons.

Notes:

- If a column has thousands of categories (like a zip code), one-hot becomes very wide. In that case, **embeddings** are used, which map each category to a short learned vector instead.
- If the categories have a natural order (small $<$ medium $<$ large), encoding them as 1, 2, 3 can be reasonable, because the ordering is real.

Why Each Feature Gets One Input Slot Instead of Feeding All Features into One Neuron

The input layer is not made of computing neurons. It is just the input vector, with one slot per value, so "one neuron per feature" simply means "one slot per number." A neuron can take multiple inputs, but that happens in the next layer, not in the input layer.

Feeding all features into one neuron is possible, but that neuron is a hidden-layer neuron, and it has two consequences:

1. **It compresses the data.** With 3 features, one neuron computes $z = w_1x_1 + w_2x_2 + w_3x_3 + b$ and outputs a single scalar. Everything the next layer sees is that one number, so many different feature combinations collapse to the same value and information is lost. For example, with $w = (1, 1, 1)$, the rows $(1, 2, 3)$ and $(3, 2, 1)$ are indistinguishable.
2. **It limits what the network can learn.** The network can only use one weighted combination of the features. A layer with several neurons learns several different combinations (each with its own weight vector), and later layers can combine those into more complex patterns. One neuron gives only a single "view" of the data.

So the structure is:

- **Input layer:** holds the raw values, 3 slots, no computation, nothing lost.
- **First hidden layer:** every neuron takes all 3 features as input, and the number of neurons (a free choice) sets how many combinations are learned.

The input layer must keep the features separate so that the hidden neurons can each apply their own weights to each individual feature.

Weight Matrix Shapes: Example $4 \to 4 \to 3 \to 1$

Consider a dataset with 1000 rows and 5 columns (4 numeric features, 1 label), and a network with architecture $4 \to 4 \to 3 \to 1$.

Hidden layer 1. The weight matrix has shape (4×4) using the convention W of shape $(n_{\text{out}} \times n_{\text{in}})$. The layer has 4 neurons ($n_{\text{out}} = 4$) and receives the 4 input features ($n_{\text{in}} = 4$). Each row of W holds one neuron's 4 weights, and the bias b has length 4. The layer computes $z = Wx + b$, with W (4×4) times x (4×1) giving z (4×1).

Parameters for the whole architecture:

- Hidden layer 1: $W^{(1)}$ is 4×4, $b^{(1)}$ has length $4 \to 16 + 4 = 20$ parameters
- Hidden layer 2: $W^{(2)}$ is 3×4, $b^{(2)}$ has length $3 \to 12 + 3 = 15$ parameters
- Output layer: $W^{(3)}$ is 1×3, $b^{(3)}$ has length $1 \to 3 + 1 = 4$ parameters

Total: 39 parameters.

Weight Matrix Shape Convention: $(n_{\text{out}} \times n_{\text{in}})$ vs. $(n_{\text{in}} \times n_{\text{out}})$

Why is $W^{(2)}$ written as 3×4 and not 4×3? It depends on the convention. Here we use $(n_{\text{out}} \times n_{\text{in}})$, where each row holds one neuron's weights.

Hidden layer 2 has 3 neurons and receives 4 inputs:

- Each neuron needs 4 weights (one per input), so each neuron is a row of length 4.
- There are 3 neurons, so there are 3 rows.
- $W^{(2)}$ is therefore 3×4.

This makes the math work with the column-vector form $z = Wa + b$: (3×4) times (4×1) gives (3×1), a vector with one value per neuron.

The alternative 4×3 is the other common convention, $(n_{\text{in}} \times n_{\text{out}})$, where each *column* holds one neuron's weights. It is valid, but the computation is then written as $z = a^\top W + b$, i.e. (1×4) times (4×3) gives (1×3). PyTorch's `nn.Linear` stores the weight as $(n_{\text{out}} \times n_{\text{in}})$, while Keras's `Dense` stores it as $(n_{\text{in}} \times n_{\text{out}})$. The two are transposes of each other, with the same 12 numbers.

Whichever convention is chosen, it must be used consistently so that the shapes multiply correctly. In general, a layer's weight shape is (neurons in this layer $\times$ neurons in the previous layer) under the first convention.

Week 1: The Neuron and the MLP

Scalar. One single number, such as 3 or -0.5.

Vector. A list of numbers written top to bottom. The symbol $\mathbb{R}^d$ means "a list of d ordinary (real) numbers."

Matrix. A grid of numbers. The symbol $W \in \mathbb{R}^{m \times d}$ means W has m rows and d columns.

Dot product. Multiply matching entries of two lists and add the results. For $(1, 2, 3)$ and $(4, 5, 6)$ it is $1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$.

Matrix times vector. Take the dot product of each row of the matrix with the vector. The answers form a new vector with one entry per row.

Transpose, $\top$. Flip a matrix so rows become columns. A column vector x becomes a row $x^\top$.

Derivative. "If I nudge the input a tiny bit, how fast does the output change?" It is the slope of the graph.

Partial derivative, $\partial L / \partial w$. The same idea when there are many inputs: nudge only w and hold everything else still.

Gradient, ∇. The list of all partial derivatives. $\nabla_W L$ means "how L changes when each entry of W is nudged."

Chain rule. If A affects B and B affects C, then "how fast C changes with A" equals "how fast C changes with B" times "how fast B changes with A."

Loss, L. One number that says how wrong the network's answer is. Smaller is better. Training means making the loss smaller.

Logarithm. In these notes log and ln both mean the natural logarithm (base $e \approx 2.718$). Useful facts: $\log 1 = 0$, and the log of a number between 0 and 1 is negative.

Block A: The slides

Slide 2: Today

What the slide shows. Three headed lists in two columns.

Theory (left column).

- The neuron as an affine map plus a nonlinearity: a neuron does a straight-line style calculation, then bends the result.
- Why depth needs activations (XOR): stacking layers only helps if something bends in between. XOR is the test case.
- Fully connected layers and parameter shapes: how the weight grids are sized and how to count them.
- MSE, binary cross-entropy, softmax cross-entropy: the three standard ways to score a wrong answer.
- Computation graphs and the chain rule: drawing a calculation as boxes and arrows so derivatives can be found step by step.
- Backpropagation in scalars, then matrices: first with single numbers, then with grids.

Practice in this room (right column, top). One logistic neuron by hand; a $2 \to 2 \to 1$ ReLU network forward and backward; a "dead ReLU" unit and what its gradient does; four problems whose solutions are linked from each slide.

After class (right column, bottom). Read Goodfellow Chapter 6 and Prince Chapters 3–4; do Home Assignment W1 (a NumPy MLP with an autograd check); Quiz 1 is this week.

Slide 3: Learning objectives

What the slide shows. Five things you should be able to do by the end of the lecture, and two course outcomes (CO1 and CO2) that the lecture touches.

1. Write the forward map of a multilayer perceptron, including every shape. "Forward map" means the recipe that turns an input into an output. "Every shape" means the number of rows and columns of every grid and list.
2. Differentiate sigmoid, tanh, and ReLU, and say where each gradient vanishes (becomes almost or exactly zero).
3. Derive $\partial L/\partial z = \hat{y} - y$ for binary cross-entropy with a sigmoid (slide 9 does this).
4. Backpropagate a scalar loss through one hidden layer using outer products (slides 25 to 28).
5. Match the column-vector formulas on the board to the row-batch layout in the NumPy assignment (slide 30).

CO1 is "explain the primitives" (the basic building blocks). CO2 is "the backprop you will code."

Slide 4: Notation used on the board

What the slide shows. A table of every symbol used in the lecture. Here it is with a plain-English column added.

Symbol	Meaning	Plain English
$x = a^{(0)} \in \mathbb{R}^d$	one input, a column	The input list of d numbers. It is also called "layer 0's activation."
$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$	weights of layer ℓ (out $\times$ in)	A grid with one row per neuron in this layer and one column per number coming in.
$b^{(\ell)} \in \mathbb{R}^{n_\ell}$	bias	One adjustable offset per neuron.
$z^{(\ell)}$	$W^{(\ell)}a^{(\ell-1)}+b^{(\ell)}$, the pre-activation	The raw score before the bending step.
$a^{(\ell)} = f(z^{(\ell)})$	activation; f applied elementwise	The score after the bending step, applied to each entry separately.
$\hat{y} = a^{(L)}$	prediction	The last layer's output is the network's answer.
L	loss on one example	How wrong the answer is for one example, unless a mean is stated.
$\delta^{(\ell)} = \nabla_{z^{(\ell)}} L$	backprop signal at layer ℓ	How sensitive the loss is to each raw score in layer ℓ.
$\odot$	elementwise (Hadamard) product	Multiply two lists entry by entry.

Reading the table.

- ℓ (a script "l") is the layer number. Layer 0 is the input. Layer L is the last layer. n_ℓ is how many neurons layer ℓ has.
- The superscript in parentheses, like $W^{(2)}$, is a label for the layer, not a power.
- $\hat{y}$ is read "y-hat." The hat means "the network's guess" as opposed to y, the true answer.
- **Hadamard example.** $(1, 2, 3) \odot (4, 5, 6) = (4, 10, 18)$. This is different from the dot product, which adds the results up.
- The slide warns that the home assignment stores a batch of rows and uses $Z = XW + b$, so its W is the transpose of the board's W.

Slide 5: An affine score

What the slide shows. The calculation inside a single neuron, three bullet points about it, and one equation showing that stacked straight-line maps collapse.

Equation 1: the neuron.

$$z = w^\top x + b, \qquad a = f(z).$$

Symbols.

- $x \in \mathbb{R}^d$: the input, a list of d numbers.
- $w \in \mathbb{R}^d$: the weights, one adjustable number per input.
- $b \in \mathbb{R}$: the bias, one adjustable number that shifts the score up or down.
- z: the score (also called the pre-activation).
- f: the activation function, which takes one number and returns one number.
- a: the neuron's final output.

What it says. Multiply each input by its weight, add everything up, add the bias, and call that the score z. Then run the score through a bending function f ("squash it") to get the output a.

Example. Let $w = (2, -1)$, $x = (3, 4)$, $b = 1$. Then $z = 2 \cdot 3 + (-1) \cdot 4 + 1 = 3$.

The bullets.

- For one input ($d = 1$) the score is $z = wx + b$, which is a straight line.
- "Affine" means "a straight-line style formula": multiply, then add a constant. The graph of z against any one input is a straight line.
- f is the only place a curve can enter. If f does nothing (the identity function), the whole model stays straight-line style.

Equation 2: stacking affine maps collapses.

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Symbols. W_1, b_1 are the weights and bias of the first layer; W_2, b_2 belong to the second layer; x is the input.

What it says. Feed the output of one affine layer into a second affine layer (no bending in between) and the result is the same as a single affine layer whose weights are the product W_2W_1 and whose bias is $W_2b_1 + b_2$. Two layers did the work of one. Extra layers add nothing unless something nonlinear sits between them.

Example with single numbers. Take $W_1 = 2$, $b_1 = 1$, $W_2 = 3$, $b_2 = 4$. Then $3(2x + 1) + 4 = 6x + 7$. The combined weight is $W_2W_1 = 6$ and the combined bias is $W_2b_1 + b_2 = 3 + 4 = 7$. At $x = 5$ both ways give 37.

Slide 6: Logistic regression is a one-neuron network

What the slide shows. The simplest neural network: one neuron with a sigmoid activation, trained with binary cross-entropy. It also announces two facts that will be proved and then used all semester.

Equation 1: the sigmoid.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1), \qquad \hat{y} = \sigma(w^\top x + b).$$

Symbols. z is any real number; $e \approx 2.718$; σ (sigma) is the sigmoid function; $\hat{y}$ is the network's output; w, x, b are as on slide 5.

What it says. The sigmoid takes any number, however large or small, and squeezes it into the interval between 0 and 1. Very negative z gives something near 0, $z = 0$ gives exactly $1/2$, and very positive z gives something near 1. That makes the output readable as a probability: "the chance that the label is 1."

Equation 2: binary cross-entropy.

$$L = -\Big(y \log \hat{y} + (1 - y) \log(1 - \hat{y})\Big).$$

Symbols. $y \in \{0, 1\}$ is the true label; $\hat{y}$ is the predicted probability that the label is 1; L is the loss.

What it says. Only one of the two terms is ever active. If $y = 1$ the loss is $-\log \hat{y}$. If $y = 0$ the loss is $-\log(1 - \hat{y})$. In both cases the loss is small when the network puts high probability on the true answer and huge when it is confidently wrong.

Example. With $y = 1$: if $\hat{y} = 0.9$ then $L = -\log 0.9 \approx 0.105$; if $\hat{y} = 0.1$ then $L = -\log 0.1 \approx 2.303$.

The two facts to be proved.

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big), \qquad \frac{\partial L}{\partial z} = \hat{y} - y.$$

The first is proved on slide 11, the second on slide 9. The slide says the second identity is why this pair is the default binary head: the factor σ' cancels, leaving the cleanest possible error signal.

Slide 7: Worked example, logistic forward pass

What the slide shows. Numbers for one neuron with one input: $w = 2$, $b = -2$, $x = 1$, $y = 1$.

$$z = wx + b = 2 \cdot 1 - 2 = 0, \qquad \hat{y} = \sigma(0) = \tfrac{1}{2}, \qquad L = -\log \tfrac{1}{2} = \ln 2 \approx 0.693.$$

Symbols. w weight, b bias, x input, y true label, z score, $\hat{y}$ prediction, L loss.

What it says. The score is exactly zero, so the sigmoid outputs exactly one half. A one-half prediction means "I have no idea." The loss is then $\ln 2$ whichever label is correct.

The "Reading the number" box. $\hat{y} = \frac{1}{2}$ is the prediction you would make with no information, and $\ln 2$ is the loss of a coin flip. On a balanced binary problem (about half the labels are 0 and half are 1), a randomly initialized network should start with a loss near 0.693. This is a quick sanity check: if your very first loss is wildly different, suspect a bug.

Slide 8: Worked example, logistic backward pass

What the slide shows. Using the same numbers ($\hat{y} = \frac{1}{2}$, $y = 1$, $x = 1$), it finds the gradients and takes one learning step.

$$\frac{\partial L}{\partial z} = \hat{y} - y = \tfrac{1}{2} - 1 = -\tfrac{1}{2}.$$

What it says. The error signal at the score is $-\frac{1}{2}$. The negative sign means "the score is too low; raise it."

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = -\tfrac{1}{2} \cdot 1 = -\tfrac{1}{2}, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = -\tfrac{1}{2}.$$

Why. This is the chain rule through $z = wx + b$. Nudging w by a small amount changes z by x times that amount, so $\partial z/\partial w = x$. Nudging b changes z by exactly that amount, so $\partial z/\partial b = 1$.

$$w \leftarrow w - \eta\Big(-\tfrac{1}{2}\Big) = 2 + \tfrac{\eta}{2}.$$

Symbols. The arrow $\leftarrow$ means "replace the old value with." η (eta) is the learning rate, a small positive number that sets the step size.

What it says. Move the weight a small step opposite to its gradient, which is the downhill direction for the loss. The gradient is negative, so the weight goes up. A larger weight makes $z = wx + b$ larger when $x = 1$, so $\hat{y}$ moves toward the label 1. The bias gets the same treatment: $b \leftarrow -2 + \eta/2$.

Slide 9: Where $\partial L/\partial z = \hat{y} - y$ comes from

What the slide shows. The derivation in two steps: differentiate the loss with respect to $\hat{y}$ first, then multiply by the sigmoid's derivative.

Step 1.

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}.$$

Symbols. L is the binary cross-entropy from slide 6; $\hat{y} = \sigma(z)$.

What happens. The derivative of $-y \log \hat{y}$ is $-y/\hat{y}$. The derivative of $-(1-y)\log(1-\hat{y})$ is $+(1-y)/(1-\hat{y})$ (the extra minus sign comes from the inside of $1-\hat{y}$). Putting both over the common bottom $\hat{y}(1-\hat{y})$ gives top $-y(1-\hat{y}) + (1-y)\hat{y} = -y + y\hat{y} + \hat{y} - y\hat{y} = \hat{y} - y$.

Step 2.

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y.$$

What happens. By the chain rule, $\partial L/\partial z = (\partial L/\partial \hat{y}) \cdot (\partial \hat{y}/\partial z)$, and $\partial \hat{y}/\partial z = \sigma'(z) = \hat{y}(1-\hat{y})$. That factor is exactly the bottom of the fraction from step 1, so the two cancel.

Contrast with squared error. If the loss were $L = \frac{1}{2}(\hat{y}-y)^2$, the same chain would give $(\hat{y}-y)\sigma'(z)$, with the extra $\sigma'(z)$ left over. In the flat ends of the sigmoid ("saturated tails") $\sigma' \approx 0$.

Example. Let $y = 1$ and the score be $z = -8$, so the network is confidently wrong ($\hat{y} \approx 0.000335$). Squared error gives a gradient of about $(-1)(0.000335) \approx -0.000335$, so almost nothing is learned. Binary cross-entropy gives $\hat{y} - y \approx -1$, a strong correction. That is the reason the binary head is sigmoid plus cross-entropy.

Slide 10: Three activations and their derivatives

What the slide shows. A table of three activation functions, their derivatives, and where each derivative is tiny.

Name	Formula $f(z)$	Derivative $f'(z)$	Where the gradient is tiny
Sigmoid	$\dfrac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	when $\lvert z\rvert$ is large; the largest value of σ' is $\frac{1}{4}$, at $z = 0$
tanh	$\tanh z$	$1 - \tanh^2 z$	when $\lvert z\rvert$ is large; range is $(-1, 1)$
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$	on the entire half-line $z < 0$ (a dead unit)

Symbols. $|z|$ is the size of z ignoring its sign. $\max(0, z)$ means "the larger of 0 and z." $\mathbf{1}[z > 0]$ is an indicator: it equals 1 when the statement in brackets is true and 0 otherwise.

Plain English.

- **Sigmoid** squeezes numbers into $(0, 1)$. It flattens out at both ends, so a very large or very negative score gives almost no gradient.
- **tanh** is a similar S-shape that squeezes into $(-1, 1)$, centered on zero. Its slope is 1 at $z = 0$ and flattens at both ends.
- **ReLU** (rectified linear unit) passes positive numbers through unchanged and replaces negative numbers with 0. For example $\text{ReLU}(2) = 2$ and $\text{ReLU}(-3) = 0$. Its slope is 1 for $z > 0$ and 0 for $z < 0$, so a unit whose score is negative sends no gradient back.

The three bullets under the table.

- At exactly $z = 0$ the ReLU has a corner, so it is not classically differentiable. Any number in $[0, 1]$ is an acceptable "subgradient"; in code, use 0.
- **Leaky ReLU**, $f(z) = \max(\alpha z, z)$ with $\alpha = 0.01$, keeps a thin slope on the negative side so a unit is never completely silent.
- **Softmax** is a map from K numbers to K numbers. It is used on a whole output layer, not on one hidden unit.

Slide 11: Sigmoid derivative, in one line of algebra

What the slide shows. A proof that $\sigma'(z) = \sigma(z)(1-\sigma(z))$ and some exact values.

$$\sigma(z) = (1+e^{-z})^{-1}, \qquad \sigma'(z) = (1+e^{-z})^{-2} \cdot e^{-z} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

Symbols. σ' is the derivative of σ; e^{-z} is the number e raised to the power $-z$.

What happens, step by step.

1. Write the sigmoid as "one over $(1 + e^{-z})$", i.e. $(1 + e^{-z})^{-1}$.
2. The derivative of u^{-1} is $-u^{-2}$ times the derivative of u. Here $u = 1 + e^{-z}$, whose derivative is $-e^{-z}$. The two minus signs cancel, leaving $(1 + e^{-z})^{-2}e^{-z}$.
3. Split that into a product of two fractions. The first is $\sigma(z)$.
4. The second fraction equals $1 - \sigma(z)$, because $\dfrac{e^{-z}}{1+e^{-z}} = \dfrac{(1+e^{-z})-1}{1+e^{-z}} = 1 - \dfrac{1}{1+e^{-z}}$.

Useful exact values.

$$\sigma(0) = \tfrac{1}{2}, \qquad \sigma'(0) = \tfrac{1}{4}, \qquad \sigma'(z) \le \tfrac{1}{4} \text{ for every } z.$$

What it says. The slope of the sigmoid is never bigger than a quarter. In a deep stack of sigmoid layers, each layer multiplies the backward signal by at most 1/4. After ten layers the factor can be as small as $(1/4)^{10} = 1/1{,}048{,}576$, under one in a million. That shrinking is one reason early deep networks were hard to train. ReLU's slope on its active side is exactly 1, so it does not shrink the signal by itself.

Slide 12: XOR is not linearly separable

What the slide shows. A proof by contradiction that no straight-line rule can compute XOR. The slide has no picture, so here is one in words: put the four inputs at the corners of a square. The two class-1 points $(1, 0)$ and $(0, 1)$ sit at opposite corners, and the two class-0 points $(0, 0)$ and $(1, 1)$ sit at the other two opposite corners. No single straight line can put the class-1 corners on one side and the class-0 corners on the other.

Labels. $(0, 0) \mapsto 0$, $(1, 0) \mapsto 1$, $(0, 1) \mapsto 1$, $(1, 1) \mapsto 0$. (XOR is "exactly one of the two inputs is on.")

The assumption. Suppose some weights w_1, w_2 and bias b make the score $w^\top x + b$ positive on class 1 and negative on class 0. Checking the four points in turn gives four inequalities:

$$\begin{aligned} b &< 0 \quad \text{(point } (0,0)\text{, class 0)}, \\ w_1 + b &> 0 \quad \text{(point } (1,0)\text{, class 1)}, \\ w_2 + b &> 0 \quad \text{(point } (0,1)\text{, class 1)}, \\ w_1 + w_2 + b &< 0 \quad \text{(point } (1,1)\text{, class 0)}. \end{aligned}$$

The contradiction. The middle two lines give $w_1 > -b$ and $w_2 > -b$. Since $b < 0$, we know $-b > 0$. Add the two and then add b:

$$w_1 + w_2 + b > (-b) + (-b) + b = -b > 0.$$

But the last inequality says $w_1 + w_2 + b < 0$. A number cannot be both positive and negative, so the assumption was impossible.

What it says. No straight-line score separates XOR. A network with no hidden nonlinearity is one straight-line score, so it cannot represent XOR.

Slide 13: A two-unit ReLU net that realizes XOR

What the slide shows. A tiny network with two hidden ReLU units that gets all four XOR points exactly right, and a table that checks it. The output has no sigmoid on it ("no extra squashing"), and the slide calls the output y.

$$z^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \qquad a = \mathrm{ReLU}\big(z^{(1)}\big), \qquad y = a_1 - 2a_2.$$

Symbols. x_1, x_2 are the two inputs; $z^{(1)}$ is the pair of hidden scores; a_1, a_2 are the two hidden outputs; y is the network's output.

What it says, unit by unit. Multiplying out the matrix gives $z_1 = x_1 + x_2$ and $z_2 = x_1 + x_2 - 1$.

- The first unit just counts how many inputs are on: 0, 1, or 2.
- The second unit is positive only when both inputs are on ($z_2 = 1$) and is cut to 0 otherwise by the ReLU. It is an "AND detector."
- The output is $a_1 - 2a_2$. When exactly one input is on, $a_1 = 1$ and $a_2 = 0$, so the output is 1. When both are on, $a_1 = 2$ and $a_2 = 1$, so the output is $2 - 2 = 0$.

x_1	x_2	z_1	z_2	a_1	a_2	y
0	0	0	−1	0	0	0
1	0	1	0	1	0	1
0	1	1	0	1	0	1
1	1	2	1	2	1	0

The outputs $0, 1, 1, 0$ match the XOR labels. The slide's closing line is "Depth is doing something a line cannot do": the hidden layer plus the ReLU bend makes XOR possible.

Slide 14: The network, drawn

What the slide shows (the picture). Three columns of circles. On the left are two input circles, x_1 above x_2. In the middle are two hidden circles, h_1 above h_2. On the right is one output circle, $\hat{y}$. Arrows run from each input to each hidden circle: $x_1 \to h_1$ and $x_2 \to h_2$ are straight across, while $x_1 \to h_2$ and $x_2 \to h_1$ are diagonals that cross each other in the middle. Then arrows run from h_1 and h_2 to $\hat{y}$.

What the picture means.

- Every hidden unit sees every input. That is a *fully connected* layer.
- Each arrow carries one weight. Biases are extra arrows from a constant 1; they are left out of the picture but kept in the formulas.
- **Parameter count.** $W^{(1)}$ has $2 \cdot 2 = 4$ entries (four arrows), $b^{(1)}$ has 2, $W^{(2)}$ has 2 (two arrows to the output), and $b^{(2)}$ has 1. Total: $4 + 2 + 2 + 1 = 9$.

Slide 15: Layer recursion

What the slide shows. The rule that builds a whole network out of identical layer steps.

$$a^{(0)} = x, \qquad z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = f^{(\ell)}\big(z^{(\ell)}\big), \qquad \ell = 1, \ldots, L-1.$$

Symbols. ℓ is the layer number; L is the number of the last layer; $f^{(\ell)}$ is the activation used in layer ℓ (it can differ from layer to layer).

What it says. Start with the input as "layer 0." To get any layer's output, take the previous layer's output, multiply by this layer's weight grid, add this layer's bias, and then bend the result with this layer's activation. Repeat. (The slide runs this rule up to layer $L-1$, the hidden layers, and treats the output layer separately; the lecture notes on page 3 write the same rule for all layers up to L.)

The output layer depends on the task.

- Regression (predicting a number): $f^{(L)}$ is the identity, and the loss is squared error.
- Binary classification (two classes): $f^{(L)} = \sigma$, and the loss is binary cross-entropy.
- K-class classification: $f^{(L)} =$ softmax, and the loss is cross-entropy.

Shapes for one example. If layer ℓ has n_ℓ neurons, then $W^{(\ell)}$ is $n_\ell \times n_{\ell-1}$ and $b^{(\ell)}$, $z^{(\ell)}$, $a^{(\ell)}$ are all $n_\ell \times 1$. A quick check: $W^{(\ell)}$ $(n_\ell \times n_{\ell-1})$ times $a^{(\ell-1)}$ $(n_{\ell-1} \times 1)$ gives $n_\ell \times 1$, which is the shape of $b^{(\ell)}$, so the addition works.

Slide 16: Counting parameters

What the slide shows. A formula for how many adjustable numbers a layer has, and a worked example.

$$n_{\text{out}} \cdot n_{\text{in}} + n_{\text{out}}.$$

Symbols. n_{in} is how many numbers go into the layer; n_{out} is how many come out.

What it says. The weight grid has $n_{\text{out}} \times n_{\text{in}}$ entries, and the bias list has n_{out} entries. Add them.

Example: $784 \to 128 \to 64 \to 10$**.** (A small MNIST-style network: 784 is a 28×28 pixel image flattened into one list, and 10 is the number of digit classes.)

$$784 \cdot 128 + 128 = 100{,}480,$$
$$128 \cdot 64 + 64 = 8{,}256,$$
$$64 \cdot 10 + 10 = 650.$$

The total is $100{,}480 + 8{,}256 + 650 = 109{,}386$ parameters. The first layer holds about 92% of them because the input is so wide. Biases are cheap. The slide says this is a Quiz-style question: write the formula, then the arithmetic.

Slide 17: Three losses

What the slide shows. A table with the three standard loss functions and the gradient to remember for each.

Task	Loss on one example	Gradient to remember
Regression, $\hat{y} \in \mathbb{R}$	$L = \frac{1}{2}(\hat{y} - y)^2$	$\partial L/\partial \hat{y} = \hat{y} - y$
Binary, $\hat{y} = \sigma(z)$	$L = -\big(y \log \hat{y} + (1-y)\log(1-\hat{y})\big)$	$\partial L/\partial z = \hat{y} - y$
K classes	$L = -\sum_{k=1}^{K} y_k \log p_k$	$\nabla_z L = p - y$

Symbols. In the K-class row, $p = \text{softmax}(z)$ and y is one-hot. In general, y is the true answer; $\hat{y}$ the prediction; z the raw score; p the vector of predicted probabilities; y_k and p_k are the entries for class k. "One-hot" means a list that is 1 in the true class's position and 0 everywhere else, such as $(0, 1, 0)$.

What each row says.

- **Regression:** the loss is half the squared gap between guess and truth. The $\frac{1}{2}$ cancels the 2 that appears when differentiating, so the gradient is just the gap $\hat{y} - y$.
- **Binary:** the loss from slide 6. The gradient with respect to the *score* z is again the plain gap $\hat{y} - y$ (slide 9).
- K **classes:** because y is one-hot, the sum has only one nonzero term. The loss is $-\log$ of the probability given to the true class. The gradient with respect to the scores is again "prediction minus truth," $p - y$ (proved on lecture-note page 4).

Batch warning. A mean over a batch of n examples multiplies every parameter gradient by $1/n$. If the code uses a mean but the derivative was written for a sum (or the reverse), the effective learning rate silently changes by a factor of n. For example, with $n = 4$ a forgotten $1/4$ makes every step four times too big.

Slide 18: Softmax, with fractions

What the slide shows. The softmax formula and a clean worked example.

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Symbols. $z = (z_1, \ldots, z_K)$ are the raw class scores, called *logits*; p_i is the probability for class i; $\sum_{j=1}^{K}$ means "add up the terms for $j = 1$ through K."

What it says. Raise e to each score, so every number becomes positive. Then divide each by the total so they add to 1. The result is a list of probabilities, and a higher score gets a higher probability.

Example. Logits $z = (\ln 2, 0, 0)$. Then $e^z = (2, 1, 1)$, the total (normalizer) is 4, and

$$p = \left(\tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{4}\right).$$

If the true class is class 1, then $y = (1, 0, 0)$ and

$$L = -\log \tfrac{1}{2} = \ln 2, \qquad \nabla_z L = p - y = \left(-\tfrac{1}{2},\ \tfrac{1}{4},\ \tfrac{1}{4}\right).$$

The sum-to-zero check. The entries of $p - y$ add to 0, because p adds to 1 and a one-hot y adds to 1, and $1 - 1 = 0$. If your coded gradient for the output layer does not add to zero, there is a bug. The slide calls this "a useful debugging test."

Slide 19: Stable binary loss (what you will code)

What the slide shows. Two tricks that keep the computer from overflowing or hitting $\log 0$.

The problem. If you first compute $\sigma(z)$ and then take $\log \hat{y}$, large scores break things. For very negative z the term e^{-z} is huge and can overflow (ordinary double-precision numbers overflow near e^{709}). For very positive z, $\sigma(z)$ rounds to exactly 1, and then $\log(1 - \hat{y}) = \log 0$ is minus infinity.

The fix for binary loss.

$$L(z, y) = \max(z, 0) - yz + \log\bigl(1 + e^{-|z|}\bigr).$$

Symbols. z is the raw score; $y \in \{0, 1\}$ the label; $|z|$ the size of z with the sign dropped.

What it says. This is exactly the same binary cross-entropy, rewritten so that the only exponential is $e^{-|z|}$, which is never larger than 1 and so cannot overflow. **Check:** at $z = 0$, $y = 1$ it gives $0 - 0 + \log 2 = \ln 2$, matching slide 7. At $z = 2$, $y = 1$ it gives $2 - 2 + \log(1 + e^{-2}) \approx 0.1269$, and the direct formula $-\log \sigma(2)$ gives the same 0.1269.

The home assignment uses a simpler safeguard, clipping $\hat{y}$ into $[\varepsilon, 1-\varepsilon]$ (never exactly 0 or 1). PyTorch's `binary_cross_entropy_with_logits` uses the identity above.

The fix for softmax.

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad m = \max_j z_j.$$

Symbols. m is the largest logit.

What it says. Subtract the biggest score from every score before exponentiating. This does not change the probabilities, because $e^{z_i - m} = e^{z_i} e^{-m}$ and the factor e^{-m} appears in both the top and the bottom and cancels. It does keep every exponent at 0 or below, so no exponential can exceed 1.

Slide 20: Running example, a $2 \to 2 \to 1$ net

What the slide shows. The numbers for the example used from here through slide 29. One example, with both hidden units "alive" (positive score), so every gate in the backward pass is visible.

$$x = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad y = 1, \quad W^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ -1/2 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 1/2 & -1 \end{pmatrix}, \quad b^{(2)} = 0.$$

Setup. The hidden activation is ReLU, the output activation is sigmoid, and the loss is binary cross-entropy. This is the network drawn on slide 14, with $4 + 2 + 2 + 1 = 9$ parameters. The plan: compute $z^{(1)}, a^{(1)}, z^{(2)}, \hat{y}, L$ exactly, then every parameter gradient, then one gradient step.

Slide 21: Forward pass, layer 1

What the slide shows.

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1/2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}.$$

Step by step. Row 1 dotted with x: $1 \cdot 1 + (-1) \cdot 0 = 1$; plus bias 0 gives 1. Row 2 dotted with x: $1 \cdot 1 + 1 \cdot 0 = 1$; plus bias $-\frac{1}{2}$ gives $\frac{1}{2}$.

Both scores are positive, so ReLU changes nothing:

$$a^{(1)} = \mathrm{ReLU}\big(z^{(1)}\big) = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, \qquad \mathrm{ReLU}'\big(z^{(1)}\big) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Why keep $z^{(1)}$. The backward pass needs the mask $\mathbf{1}[z^{(1)} > 0]$. The activation $a^{(1)}$ alone cannot rebuild that mask once an entry is zero, because "the score was negative" and "the score was exactly zero" both produce an activation of 0. So the score $z^{(1)}$ must be saved.

Slide 22: Forward pass, layer 2 and the loss

What the slide shows.

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = \tfrac{1}{2} \cdot 1 + (-1) \cdot \tfrac{1}{2} + 0 = 0, \qquad \hat{y} = \sigma(0) = \tfrac{1}{2}, \qquad L = -\log \tfrac{1}{2} = \ln 2.$$

What it says. The output score is exactly 0, so the prediction is $\frac{1}{2}$ and the loss is $\ln 2$. The label is $y = 1$, so only the $-\log \hat{y}$ term of the loss is active.

The "Cache for backprop" box. Store $\big(x,\ z^{(1)},\ a^{(1)},\ z^{(2)}\big)$. The home assignment returns the same four arrays, batched. Anything that can be recomputed from these is optional; the pre-activations are not optional for ReLU.

Slide 23: Scalar computation graph

What the slide shows. A warm-up with no network: just single numbers wired together. Let $w = 2$, $x = 3$, $y = 4$ and

$$u = wx, \qquad r = u - y, \qquad L = \tfrac{1}{2}r^2.$$

Forward pass (left to right). $u = 2 \cdot 3 = 6$, then $r = 6 - 4 = 2$, then $L = \frac{1}{2} \cdot 4 = 2$.

Backward pass (right to left). At each step write the local derivative, then multiply along the path:

$$\begin{aligned} \frac{\partial L}{\partial r} &= r = 2, \\ \frac{\partial L}{\partial u} &= \frac{\partial L}{\partial r} \cdot 1 = 2, \\ \frac{\partial L}{\partial w} &= \frac{\partial L}{\partial u} \cdot x = 2 \cdot 3 = 6, \\ \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial u} \cdot w = 2 \cdot 2 = 4. \end{aligned}$$

Symbols. u and r are in-between values; each fraction reads "how much does L change per unit change in this variable."

What it says. Each line uses one simple local rule. $L = \frac{1}{2}r^2$ has slope r. $r = u - y$ has slope 1 in u. $u = wx$ has slope x in w and slope w in x. Multiplying slopes along the path gives the slope of the whole chain. (By the same method, $\partial L/\partial y = 2 \cdot (-1) = -2$.)

Direct check. Write $L = \frac{1}{2}(wx - y)^2$ in one piece. Then $\partial L/\partial w = (wx - y)\, x = 2 \cdot 3 = 6$, the same answer.

Slide 24: The same graph, as a picture

What the slide shows (the picture). A row of four boxes joined by arrows pointing right: $w = 2 \to u = wx \to r = u - y \to L = \frac{1}{2}r^2$. A box $x = 3$ sits above the u box with an arrow pointing down into it, and a box $y = 4$ sits above the r box with an arrow pointing down into it. So each calculation box has two incoming arrows: u takes w and x, and r takes u and y.

What the picture means.

- Each arrow carries a *local* derivative: $\partial u/\partial w = x$, $\partial r/\partial u = 1$, $\partial L/\partial r = r$.
- The *global* derivative, the effect of one variable on the final loss, is the product of the local derivatives along the path from that variable to L.
- If a variable reaches the loss along two different paths, compute each path's product and *add* them. That addition is the multivariate chain rule.

Example of two paths. Let $f = w \cdot w$, treating the two copies of w as separate inputs. Each copy has slope w (the other copy), and w reaches f through both, so the total slope is $w + w = 2w$, which matches the usual derivative of w^2.

Slide 25: Outer products are the chain rule in index form

What the slide shows. The three formulas for one fully connected layer, derived from single entries.

Setup. $z = Wx + b$ with $z \in \mathbb{R}^m$, $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{m \times d}$. Suppose L depends on z and you already know $\delta = \nabla_z L$.

$$z_i = \sum_{j=1}^{d} W_{ij} x_j + b_i.$$

Symbols. z_i is the i-th score; W_{ij} is the weight in row i, column j (it connects input j to output i); x_j is the j-th input.

Differentiate one entry.

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z_i} \cdot x_j = \delta_i\, x_j.$$

What it says. The weight W_{ij} affects only the score z_i, and it does so by the amount x_j per unit of weight. So its effect on the loss is "how much the loss cares about z_i" times "the input it multiplies."

Collect all entries into three formulas.

$$\nabla_W L = \delta\, x^\top, \qquad \nabla_b L = \delta, \qquad \nabla_x L = W^\top \delta.$$

- $\delta x^\top$ is the *outer product*: a column times a row gives a grid whose entry in row i, column j is $\delta_i x_j$. **Example:** $\delta = (1, 2)^\top$ and $x = (3, 4, 5)^\top$ give $\delta x^\top = \begin{pmatrix} 3 & 4 & 5 \\ 6 & 8 & 10 \end{pmatrix}$, which has the same shape as W.
- $\nabla_b L = \delta$ because the bias adds straight into z_i with slope 1.

- $\nabla_x L = W^\top \delta$ because input x_j feeds every score z_i with weight W_{ij}, so its total effect is the sum $\sum_i W_{ij}\delta_i$.

The slide's advice: memorize these three. Every fully connected backward step in the course is this block, plus an elementwise f' when there is an activation.

Slide 26: The δ recursion

What the slide shows. How the error signal δ travels backward through the layers.

At the output (binary cross-entropy with a sigmoid):

$$\delta^{(L)} = \hat{y} - y.$$

For a hidden layer with an elementwise activation f:

$$\delta^{(\ell)} = \Big(\big(W^{(\ell+1)}\big)^\top \delta^{(\ell+1)}\Big) \odot f'\big(z^{(\ell)}\big).$$

Parameter gradients at every layer, output included:

$$\nabla_{W^{(\ell)}} L = \delta^{(\ell)}\big(a^{(\ell-1)}\big)^\top, \qquad \nabla_{b^{(\ell)}} L = \delta^{(\ell)}.$$

Symbols. $\delta^{(\ell)}$ is the error signal at layer ℓ's scores; $W^{(\ell+1)}$ is the next layer's weight grid; $\odot$ is entry-by-entry multiplication; $f'(z^{(\ell)})$ is the activation's slope at each score.

What it says. Take the error signal from the layer above, pass it back through the weights it came through (that is the transpose), and then scale each entry by the slope of that neuron's activation. If a neuron's slope is 0 (a dead ReLU), its error signal becomes 0.

Why the transpose (the slide's box). $W^{(\ell+1)}$ has shape (next $\times$ current). Multiplying by its transpose turns a list of "next" length back into a list of "current" length. The Hadamard product with f' applies the local slope of the activation at this layer.

Slide 27: Backward pass on the running example

What the slide shows. Using $\hat{y} = \frac{1}{2}$, $y = 1$, $a^{(1)} = (1, \frac{1}{2})^\top$, and $\mathrm{ReLU}'(z^{(1)}) = (1, 1)^\top$:

$$\delta^{(2)} = \hat{y} - y = -\tfrac{1}{2}.$$

$$\nabla_{W^{(2)}} L = \delta^{(2)}\big(a^{(1)}\big)^\top = -\tfrac{1}{2}\begin{pmatrix}1 & \frac{1}{2}\end{pmatrix} = \begin{pmatrix}-\frac{1}{2} & -\frac{1}{4}\end{pmatrix}, \qquad \nabla_{b^{(2)}} L = -\tfrac{1}{2}.$$

$$\delta^{(1)} = \big(W^{(2)}\big)^\top \delta^{(2)} = \begin{pmatrix}1/2\\ -1\end{pmatrix}\begin{pmatrix}-\frac{1}{2}\end{pmatrix} = \begin{pmatrix}-1/4\\ 1/2\end{pmatrix}.$$

What it says. The output error is $-\frac{1}{2}$ (the prediction is too low). The output weights get gradients proportional to the hidden outputs they multiply. The error is then sent back through $W^{(2)}$: hidden unit 1 receives $\frac{1}{2} \cdot (-\frac{1}{2}) = -\frac{1}{4}$ and hidden unit 2 receives $-1 \cdot (-\frac{1}{2}) = \frac{1}{2}$. The ReLU mask is all ones (both units are active), so $\delta^{(1)}$ passes through unchanged.

Slide 28: Parameter gradients at layer 1

What the slide shows.

$$\nabla_{W^{(1)}} L = \delta^{(1)} x^\top = \begin{pmatrix}-1/4\\ 1/2\end{pmatrix}\begin{pmatrix}1 & 0\end{pmatrix} = \begin{pmatrix}-1/4 & 0\\ 1/2 & 0\end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix}-1/4\\ 1/2\end{pmatrix}.$$

The "Read the zeros" box. Because $x_2 = 0$, the second column of $\nabla_{W^{(1)}} L$ is zero. Those two weights multiply an input that is 0, so they did not affect $z^{(1)}$ on this particular input, and a zero gradient is the correct answer. It is *not* a dead ReLU.

How to tell the two zeros apart. A zero *column* comes from a zero *input* entry. A dead ReLU zeroes a *row* of the gradient, through f', even when the input is nonzero, because the whole neuron's error signal is cut to zero. The slide says Problem 3 is that case.

Slide 29: One gradient step, then the loss falls

What the slide shows. Take one step with learning rate $\eta = \frac{1}{10}$, using new = old $-\,\eta \cdot$ gradient. "Only the nonzero gradients move."

$$W^{(2)} \leftarrow \begin{pmatrix} 11/20 & -39/40 \end{pmatrix}, \quad b^{(2)} \leftarrow \tfrac{1}{20}, \quad W^{(1)} \leftarrow \begin{pmatrix} 41/40 & -1 \\ 19/20 & 1 \end{pmatrix}, \quad b^{(1)} \leftarrow \begin{pmatrix} 1/40 \\ -11/20 \end{pmatrix}.$$

Where these come from. For example $W^{(2)}$: $\frac{1}{2} - \frac{1}{10}(-\frac{1}{2}) = \frac{1}{2} + \frac{1}{20} = \frac{11}{20}$ and $-1 - \frac{1}{10}(-\frac{1}{4}) = -1 + \frac{1}{40} = -\frac{39}{40}$. For $b^{(1)}$: $0 - \frac{1}{10}(-\frac{1}{4}) = \frac{1}{40}$ and $-\frac{1}{2} - \frac{1}{10}(\frac{1}{2}) = -\frac{11}{20}$. Lecture-note page 6 shows all of the arithmetic.

Forward pass again on the same x.

$$z^{(1)} = \begin{pmatrix} 21/20 \\ 2/5 \end{pmatrix}, \quad a^{(1)} = z^{(1)}, \quad z^{(2)} = \tfrac{19}{80} = 0.2375.$$

$$\hat{y} = \sigma(19/80) \approx 0.559, \qquad L \approx 0.581 \;\; (\text{it was } \ln 2 \approx 0.693).$$

What it says. After one small step the prediction moved from 0.50 toward the label 1, and the loss dropped from 0.693 to 0.581. That is the whole idea of training: compute the loss, compute the gradients, take a small step downhill, repeat. (Choosing η well is a Week 2 topic.)

Slide 30: From one column to a batch of rows

What the slide shows. A dictionary between the board formulas (one example, a column) and the NumPy formulas (a batch of n examples, one per row, with a mean loss).

	Board (one column)	NumPy (batch of rows)
Forward	$z = Wx + b$	$Z = XW_{\text{code}} + b$
Weights	W is (out $\times$ in)	$W_{\text{code}} = W^\top$ is (in $\times$ out)
Output error	$\delta = \hat{y} - y$	$dZ_2 = (\hat{Y} - Y)/n$
∇_W	$\delta\, a^\top$	$dW_{\text{code}} = A^\top dZ$
To the layer below	$W^\top \delta$	$dA = dZ\, W_{\text{code}}^\top$
ReLU	$\delta \odot \mathbf{1}[z > 0]$	$dZ_1 = dA \odot \mathbf{1}[Z_1 > 0]$

Symbols. $X \in \mathbb{R}^{n \times d}$ holds one input per row; Z, A are the batch versions of scores and activations (one row per example); $\hat{Y}$ and Y are the batch predictions and labels; dZ, dA, dW_{code} are the gradients (the "d" means "derivative of the loss with respect to"); n is the batch size.

What it says, row by row.

- **Why the transpose.** Writing a column equation $z = Wx + b$ for a row $x^\top$ gives $z^\top = x^\top W^\top + b^\top$. Stacking n such rows gives $Z = XW^\top + b$, so $W_{\text{code}} = W^\top$. Biases are added to every row ("broadcast").
- **Output error.** The same $\hat{y} - y$, divided by n because the loss is a mean over the batch.
- **Weight gradient.** For one example the outer product $\delta a^\top$; for a batch, $A^\top dZ$ adds up all n outer products in one multiplication (the $1/n$ is already inside dZ).
- **To the layer below and ReLU.** The same ideas as the board, in transposed layout. The mask uses the stored scores Z_1, not A.

Check on the running example ($n = 1$). The board gradient $\nabla_{W^{(2)}} L = (-\frac{1}{2}, -\frac{1}{4})$ is a row; the code gradient is that row transposed into a column. Same numbers, same linear map.

Slide 31: The training step you will implement

What the slide shows. A short block of code (a loop that runs `T` times) and three bullet points. The loop does three things on every pass:

1. Run the *forward* function on the whole batch `X` with the current parameters. It returns the predictions and a *cache*. A comment on the slide stresses that the cache holds Z_1, not only A_1 (the ReLU mask needs the scores).
2. Run a hand-written *backward* function that uses the cache, the predictions, and the labels to produce the gradients. There is no autograd here; you write the backward pass yourself.
3. Update every parameter by subtracting the learning rate `eta` times its gradient.

The three bullets.

- **Check shapes before checking values.** A bias stored with shape $(h,)$ versus $(1, h)$ is the usual first bug, because the two broadcast differently.
- **Check gradients against PyTorch on the same weights.** The relative error should be below 10^{-5} before you train.
- **A flat loss** is more often a dead hidden layer, a learning rate set to 0, or a forgotten $1/n$, than a mysterious optimizer. Optimizers are Week 2.

Slide 32: In-class problems

What the slide shows. Four problems to work on paper. Exact fractions are enough; a decimal is only needed if you choose to approximate σ.

#	Topic	Time
1	Sigmoid values and the size of σ'	4 minutes
2	Parameter count of a $4 \to 3 \to 2$ network	3 minutes
3	Forward and backward through a dead ReLU unit	12 minutes
4	Softmax cross-entropy gradient and a sum-to-zero check	6 minutes

Each problem slide has a button that jumps to its solution. The written lecture notes have eight problems with full derivations and the one-step recomputation. The slide problems line up with the notes as follows: slide Problem 1 is notes P1 (plus a $z = 8$ question that appears only on the slide), Problem 2 is notes P4, Problem 3 is notes P6, and Problem 4 is notes P5.

Slide 33: Problem 1, Sigmoid

What the slide asks.

1. Compute $\sigma(0)$ and $\sigma'(0)$ exactly.
2. Show that $\sigma'(z) \le \frac{1}{4}$ for every real z, with equality only at $z = 0$.
3. A hidden layer uses a sigmoid, and its pre-activation on a training point is $z = 8$. Give a bound on that unit's factor in the backward pass, and say what this does to learning.

Tools to use. Slide 11 (the formula $\sigma' = \sigma(1 - \sigma)$ and $\sigma(0) = \frac{1}{2}$) and the idea that a number between 0 and 1 times one minus itself is at most $\frac{1}{4}$. The answer is on slide 40.

Slide 34: Problem 2, Shapes and parameter count

What the slide asks. A network is $4 \to 3 \to 2$ with a ReLU hidden layer and logits at the output.

1. Write the shapes of $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$ in the board convention (out $\times$ in).
2. How many scalar parameters are there?
3. For a batch of 32 rows in the code convention, write the shapes of X, $W^{(1)}_{\text{code}}$, and $Z^{(1)}$.

Tools to use. Slide 15 (shapes) and slide 16 (parameter formula). The answer is on slide 41.

Slide 35: Problem 3, A dead unit

What the slide asks. Use

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad y = 0, \quad W^{(1)} = I_2, \quad b^{(1)} = \begin{pmatrix} -1/2 \\ -3/2 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & 2 \end{pmatrix}, \quad b^{(2)} = -1,$$

with a ReLU hidden layer, sigmoid output, and binary cross-entropy. (I_2 is the 2×2 identity matrix, with ones on the diagonal and zeros elsewhere; multiplying by it changes nothing.)

1. Compute $z^{(1)}, a^{(1)}, z^{(2)}, \hat{y}$, and L.
2. Compute $\delta^{(2)}$, $\nabla_{W^{(2)}} L$, $\nabla_{b^{(2)}} L$, $\delta^{(1)}$, $\nabla_{W^{(1)}} L$, and $\nabla_{b^{(1)}} L$.
3. Which hidden unit is dead, and which gradient entries are zero because of that?

Tools to use. The same steps as slides 21 to 28. The answers are on slides 42 and 43.

Slide 36: Problem 4, Softmax

What the slide asks. Logits $z = (0, \ln 3)$ and one-hot label $y = (0, 1)$.

1. Compute $p = \text{softmax}(z)$ and the cross-entropy L exactly.
2. Compute $\nabla_z L = p - y$.
3. Check that the entries of p sum to 1 and the entries of $\nabla_z L$ sum to 0.
4. For $K = 2$, softmax plus cross-entropy is the same model as a sigmoid on the logit difference. Identify the difference here and check that $\partial L / \partial z_2 = \sigma(z_2 - z_1) - 1$.

Tools to use. Slide 18. The answer is on slide 44.

Slide 37: What this week asks of you

What the slide shows. Three tasks.

- **Read**, in this order if time is short: Prince Chapters 3–4 for the mechanics, then Goodfellow Chapter 6 for the vector backprop. Nielsen Chapter 2 is an optional walkthrough with single numbers.
- **Home Assignment W1.** Build a $2 \to 16 \to 1$ network in NumPy on the two-moons data (a standard toy set of two curved, interleaved groups of points). Implement the backward pass from today's formulas, match PyTorch to a relative error below 10^{-5}, train, and draw the decision boundary (the curve where the prediction switches from one class to the other).
- **Quiz 1** can ask for a derivative, a shape, a short chain-rule step, or a verbal reason such as "why does the ReLU cache store z?" or "why does binary cross-entropy with a sigmoid simplify?" The slide says the four in-class problems are the right practice.

Slide 38: Equations worth leaving with

What the slide shows. Six formulas to remember. Here is each one in plain English.

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

Layer ℓ's raw scores are its weights times the previous layer's outputs, plus its bias.

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

The slope of the sigmoid is the output times one minus the output, never more than $\frac{1}{4}$.

$$\frac{\partial L_{\text{BCE}}}{\partial z} = \sigma(z) - y$$

For a sigmoid output with binary cross-entropy, the error signal at the score is simply prediction minus truth. (L_{BCE} is the binary cross-entropy loss.)

$$\nabla_z L_{\text{CE}} = \text{softmax}(z) - y$$

For a softmax output with cross-entropy, the error signal at the scores is again prediction minus truth, as a list. (L_{CE} is the softmax cross-entropy loss.)

$$\delta^{(\ell)} = \left(\left(W^{(\ell+1)}\right)^\top \delta^{(\ell+1)}\right) \odot f'\left(z^{(\ell)}\right)$$

The error signal at layer ℓ is the next layer's error signal sent back through the weights, then scaled neuron by neuron by the activation's slope.

$$\nabla_{W^{(\ell)}} L = \delta^{(\ell)}\left(a^{(\ell-1)}\right)^\top$$

The gradient for a weight grid is the layer's error signal (a column) times the incoming activations (a row): an outer product.

Slide 39: References

What the slide shows. Five reading sources.

- Goodfellow, Bengio, Courville, *Deep Learning*, Chapter 6 (free online at deeplearningbook.org): the vector-style backpropagation.
- Prince, *Understanding Deep Learning*, Chapters 3–4 (udlbook.github.io): the mechanics of shallow and deep networks.
- Zhang, Lipton, Li, Smola, *Dive into Deep Learning*, MLP chapter (d2l.ai): a hands-on treatment with code.
- Nielsen, *Neural Networks and Deep Learning*, Chapter 2: backpropagation with single numbers.
- Cybenko (1989) and Hornik (1991): a wide one-hidden-layer network can approximate any continuous function on a bounded, closed region of inputs to any desired accuracy.

The caveat on the slide. That last result is an *existence* result. It says a good network exists. It does not tell you how wide to make it, how to find its weights, or that training will be easy.

Slide 40: Solution 1

Part 1. $\sigma(0) = 1/2$ and $\sigma'(0) = (1/2)(1/2) = 1/4$. Since $\sigma(0) = 1/(1 + e^0) = 1/2$, the product $\sigma(0)(1 - \sigma(0))$ is a half times a half.

Part 2. Let $s = \sigma(z)$, which lies strictly between 0 and 1. Then $\sigma'(z) = s(1 - s)$. The expression $s(1-s)$ is a downward-opening parabola in s whose top is at $s = 1/2$, where it equals $1/4$. And $s = 1/2$ happens only when $z = 0$. So $\sigma'(z) \leq 1/4$ with equality only at $z = 0$. (A neat way to see the peak: $s(1 - s) = \frac{1}{4} - (s - \frac{1}{2})^2$, and a square is never negative.)

Part 3. At $z = 8$, $\sigma(8) = 1/(1 + e^{-8})$ is extremely close to 1. Then

$$\sigma'(8) = \sigma(8)\big(1 - \sigma(8)\big) < e^{-8} \approx 3.4 \times 10^{-4}.$$

Why the bound holds. $\sigma(8) < 1$, and $1 - \sigma(8) = e^{-8}/(1 + e^{-8}) < e^{-8}$. So the product is less than e^{-8}.

What it says. The backward factor through that unit is smaller than 10^{-3}. The gradient step on this unit's incoming weights is negligible: the unit is *saturated*, and it barely learns from this example.

Slide 41: Solution 2

Part 1 (board shapes, out $\times$ in). $W^{(1)} \in \mathbb{R}^{3\times 4}$, $b^{(1)} \in \mathbb{R}^3$, $W^{(2)} \in \mathbb{R}^{2\times 3}$, $b^{(2)} \in \mathbb{R}^2$. The first layer maps 4 numbers to 3, so it has 3 rows and 4 columns. The second maps 3 to 2.

Part 2 (count).

$$(3 \cdot 4 + 3) + (2 \cdot 3 + 2) = 15 + 8 = 23.$$

Part 3 (code shapes, batch of 32). $X \in \mathbb{R}^{32\times 4}$ (32 examples, 4 features each), $W^{(1)}_{\text{code}} \in \mathbb{R}^{4\times 3}$ (the transpose of 3×4), and $Z^{(1)} \in \mathbb{R}^{32\times 3}$. The shape check: $(32 \times 4)(4 \times 3) = 32 \times 3$.

Slide 42: Solution 3 (forward)

The solution as printed.

$$z^{(1)} = \begin{pmatrix}1\\1\end{pmatrix} + \begin{pmatrix}-1/2\\-3/2\end{pmatrix}, \qquad a^{(1)} = \mathrm{ReLU}\big(z^{(1)}\big), \qquad z^{(2)} = 2 \cdot \tfrac{1}{2} + 2 \cdot 0 - 1 = 0, \qquad \hat{y} = \sigma(0) = \tfrac{1}{2}.$$

A correction to check. The slide writes $z^{(1)} = (1/2,\ -1)^\top$. Do the second entry carefully: $1 + (-\frac{3}{2}) = -\frac{1}{2}$, not -1. The correct value is

$$z^{(1)} = \begin{pmatrix}1/2\\-1/2\end{pmatrix}, \qquad a^{(1)} = \mathrm{ReLU}\big(z^{(1)}\big) = \begin{pmatrix}1/2\\0\end{pmatrix}.$$

The conclusion does not change, because $-\frac{1}{2}$ is still negative, so ReLU still outputs 0, and every later number on the slide is unaffected. (Lecture-note page 8 repeats the same slip.)

Output and loss. $z^{(2)} = 2 \cdot \frac{1}{2} + 2 \cdot 0 - 1 = 0$, so $\hat{y} = \sigma(0) = \frac{1}{2}$. The label is $y = 0$, so only the $-(1-y)\log(1-\hat{y})$ term is active:

$$L = -\log \tfrac{1}{2} = \ln 2.$$

Dead unit. Unit 2 is dead on this example: its score $z^{(1)}_2$ is negative.

Slide 43: Solution 3 (backward)

Output layer.

$$\delta^{(2)} = \hat{y} - y = \tfrac{1}{2} - 0 = \tfrac{1}{2}, \qquad \nabla_{W^{(2)}} L = \tfrac{1}{2}\begin{pmatrix}1/2 & 0\end{pmatrix} = \begin{pmatrix}1/4 & 0\end{pmatrix}, \qquad \nabla_{b^{(2)}} L = \tfrac{1}{2}.$$

Here $y = 0$, so $\hat{y} - y = \frac{1}{2}$ is positive: the prediction is too high and the score should be lowered.

Sending the error back.

$$\big(W^{(2)}\big)^\top \delta^{(2)} = \begin{pmatrix}1\\1\end{pmatrix}, \qquad \mathrm{ReLU}'\big(z^{(1)}\big) = \begin{pmatrix}1\\0\end{pmatrix}, \qquad \delta^{(1)} = \begin{pmatrix}1\\1\end{pmatrix} \odot \begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}1\\0\end{pmatrix}.$$

The ReLU slope for unit 2 is 0, so unit 2's error signal is wiped out.

First-layer gradients.

$$\nabla_{W^{(1)}} L = \begin{pmatrix}1\\0\end{pmatrix}\begin{pmatrix}1 & 1\end{pmatrix} = \begin{pmatrix}1 & 1\\0 & 0\end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix}1\\0\end{pmatrix}.$$

What it says. The dead unit freezes its incoming weights (the second row of $W^{(1)}$), its bias, and its outgoing weight. The outgoing weight is frozen for a second reason too: the second entry of $\nabla_{W^{(2)}} L$ is $\delta^{(2)} a^{(1)}_2 = 0$ because $a^{(1)}_2 = 0$. Unit 1 still learns. Compare slide 28: there the zeros were a *column* (a zero input); here the zeros are a *row* (a dead neuron).

Slide 44: Solution 4

Part 1. $e^z = (e^0, e^{\ln 3}) = (1, 3)$. The normalizer is $1 + 3 = 4$, so $p = (1/4,\ 3/4)$. The true class is class 2, so

$$L = -\log \tfrac{3}{4} = \log \tfrac{4}{3}.$$

Part 2.

$$\nabla_z L = p - y = \left(\tfrac{1}{4},\ \tfrac{3}{4} - 1\right) = \left(\tfrac{1}{4},\ -\tfrac{1}{4}\right).$$

Part 3. The entries of p add to 1 ($\frac{1}{4} + \frac{3}{4}$). The entries of $\nabla_z L$ add to 0 ($\frac{1}{4} - \frac{1}{4}$).

Part 4. The logit difference is $z_2 - z_1 = \ln 3 - 0 = \ln 3$. Then

$$\sigma(\ln 3) = \frac{1}{1 + e^{-\ln 3}} = \frac{1}{1 + 1/3} = \frac{3}{4}, \qquad \sigma(\ln 3) - 1 = \tfrac{3}{4} - 1 = -\tfrac{1}{4},$$

which equals $\partial L / \partial z_2$. **What it says.** With only two classes, softmax gives class 2 the probability $\sigma(z_2 - z_1)$, exactly the sigmoid of the difference in scores. Two-class softmax and logistic regression are the same model written two ways.

Block B: The lecture notes

The lecture notes hold the same mathematics as the slides, with the algebra written out in full. Because many pages repeat slides, each section below says which slides it matches and then explains what is new.

Notes page 1: Overview, convention, the neuron, and the sigmoid derivative

The header. The title block gives the course, "Week 1 Lecture Notes," the topic ("Foundations: Neurons, MLPs, and Backpropagation"), the department, and a pointer to the slide source file and the course website.

How to use these notes. The slides are the in-class path. The notes add the written-out algebra, four fully worked examples, and eight problems with solutions. The advice is to read the theory sections before the home practice, and to attempt the problems before reading the solutions. (In the printed copy the section numbers in this paragraph show as "??" because the cross-references did not resolve.)

Suggested timing for a 3-hour meeting.

Block	Minutes
Notation and one logistic neuron	35
Activations and XOR	30
Losses and the $2 \to 2 \to 1$ forward pass	30
Chain rule and backprop on that net	40
Problems 1, 2, 6, and 7	35
Home Practice briefing	10
Total	180

Reading. Prince, Chapters 3–4; Goodfellow, Bengio, Courville, Chapter 6; optionally Nielsen, Chapter 2.

The convention. One example is a column. Layer ℓ computes

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(0)} = x,$$

with $W^{(\ell)}$ shaped (outputs $\times$ inputs). This is the same as slide 4 and slide 15; the symbols are explained there. The notes say a later section translates this into the row-batch layout used by the home practice (that is lecture-note page 6 and 7, and slide 30).

Section 1, Definition 1.1 (affine unit). For $x \in \mathbb{R}^d$, a unit with weights $w \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$ produces the pre-activation $z = w^\top x + b$ and the activation $a = f(z)$, where $f : \mathbb{R} \to \mathbb{R}$ is applied to the single number z. (This is slide 5. The notation $f : \mathbb{R} \to \mathbb{R}$ means "f takes one real number in and gives one real number out.")

The notes then repeat the collapse of two affine layers,

$$W^{(2)}\big(W^{(1)}x + b^{(1)}\big) + b^{(2)} = \big(W^{(2)}W^{(1)}\big)x + \big(W^{(2)}b^{(1)} + b^{(2)}\big),$$

and add the sentence: no choice of the two matrices can give a nonlinear decision boundary. The activation is the reason a multilayer network is not just a larger linear model.

Section 1.1, Logistic regression as a one-neuron net. Equations (1) and (2) are the sigmoid and the binary cross-entropy from slide 6, with $\hat{y} = \sigma(z)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad L(z, y) = -\Big(y \log \sigma(z) + (1 - y) \log\big(1 - \sigma(z)\big)\Big).$$

Proposition 1.1 (sigmoid derivative). $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, and $0 < \sigma'(z) \le \frac{1}{4}$, with equality only at $z = 0$.

Proof, explained. The differentiation is the one on slide 11. For the bound, let $s = \sigma(z)$, which is strictly between 0 and 1. Then both s and $1 - s$ are positive, so $\sigma'(z) = s(1 - s) > 0$. The function $s(1 - s)$ is a downward parabola with its top at $s = \frac{1}{2}$, where the value is $\frac{1}{4}$. Finally $s = \frac{1}{2}$ only when $z = 0$. The small square at the end of a proof ("□") just marks that the proof is finished.

Notes page 2: BCE through a sigmoid, stable loss, and the activation table

Proposition 1.2 (BCE through a sigmoid). If L is the binary cross-entropy (equation 2), then $\partial L/\partial z = \sigma(z) - y$.

Proof, explained. This is slide 9. The notes show one more line of the first step, combining the two fractions over a shared bottom:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} = \frac{-y(1 - \hat{y}) + (1 - y)\hat{y}}{\hat{y}(1 - \hat{y})} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}.$$

Symbols. Same as slide 9. **What it says.** The top simplifies because $-y + y\hat{y} + \hat{y} - y\hat{y} = \hat{y} - y$ (the two $y\hat{y}$ terms cancel). Multiplying by Proposition 1.1's $\hat{y}(1 - \hat{y})$ then cancels the bottom, leaving $\hat{y} - y$.

The squared-error aside. If instead $L = \frac{1}{2}(\hat{y} - y)^2$, the chain stops one step earlier and gives

$$\frac{\partial L}{\partial z} = (\hat{y} - y)\, \sigma'(z).$$

On a confidently wrong, saturated prediction ($\hat{y} \approx 0$ or 1) one has $\sigma' \approx 0$, so the update is tiny. Binary cross-entropy cancels that factor. That is why the binary head is sigmoid plus BCE, not sigmoid plus squared error.

Example 1.1 (one step of logistic regression). Exactly slides 7 and 8: $w = 2$, $b = -2$, $x = 1$, $y = 1$ gives $z = 0$, $\hat{y} = \frac{1}{2}$, $L = \ln 2$, and $\partial L/\partial z = \partial L/\partial w = \partial L/\partial b = -\frac{1}{2}$. The gradient step $w \leftarrow w - \eta\, \partial L/\partial w$ increases w, the score on this positive example goes up, and $\hat{y}$ moves toward 1.

Section 1.2, A numerically stable form of the loss. This derives the formula on slide 19. For $z \ge 0$,

$$\log \sigma(z) = -\log\big(1 + e^{-z}\big), \qquad \log\big(1 - \sigma(z)\big) = -z - \log\big(1 + e^{-z}\big).$$

Substituting these into the loss gives

$$L = (1 - y)z + \log\big(1 + e^{-z}\big).$$

Working it out. The loss is $-[\,y\log\sigma(z)+(1-y)\log(1-\sigma(z))\,]$. Plugging in gives $y\log(1+e^{-z})+(1-y)\big(z+\log(1+e^{-z})\big)$. The two log terms combine into one $\log(1+e^{-z})$, leaving $(1-y)z+\log(1+e^{-z})$.

For $z<0$ the notes say the case is symmetric after writing the exponential of $|z|$. Concretely, $\log(1+e^{-z}) = -z+\log(1+e^{z})$, and then the loss becomes $-yz+\log(1+e^{z})$. Both cases are covered by equation (3):

$$L(z,y) = \max(z,0) - yz + \log\big(1+e^{-|z|}\big).$$

What it says. For $z\geq 0$ the max term is z and $|z|=z$, giving $(1-y)z+\log(1+e^{-z})$. For $z<0$ the max term is 0 and $|z|=-z$, giving $-yz+\log(1+e^{z})$. This is what a careful implementation evaluates. Clipping $\hat{y}$ away from 0 and 1 before taking the log, as in the home practice, is a simpler approximation of the same idea.

Section 2, Activation functions (the table). Slide 10 gives the same table; the notes add the *range* (the set of possible outputs) and Leaky ReLU as a fourth row.

Name	$f(z)$	$f'(z)$	Range
Sigmoid	$\sigma(z)$ from equation (1)	$\sigma(z)(1-\sigma(z))$	$(0,1)$
tanh	$\tanh z = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$1-\tanh^2 z$	$(-1,1)$
ReLU	$\max(0,z)$	$\mathbf{1}[z>0]$	$[0,\infty)$
Leaky ReLU	$\max(\alpha z, z)$, $\alpha=0.01$	α if $z<0$, else 1	$\mathbb{R}$

Reading the range column. $(0,1)$ means every output is strictly between 0 and 1. $[0,\infty)$ means 0 or any positive number. $\mathbb{R}$ means any real number, positive or negative.

Notes page 3: Dead ReLU, universal approximation, XOR, and the MLP definition

Dead ReLU, in detail. At $z=0$ the ReLU derivative is a subgradient, meaning any number in $[0,1]$; implementations use 0. The practical consequence is one-sided: if $z<0$, the unit contributes nothing upstream. Its incoming weights, its bias, and (because its activation value is also 0) its outgoing weight all receive zero gradient from this example. Across a batch the unit can still learn from other rows. If *every* row pushes the score negative, the unit is frozen for the rest of training. That is a *dead ReLU*. Leaky ReLU replaces the zero slope by α, so a dead-looking unit still gets a small update.

Universal approximation, stated only. A single hidden layer of sigmoids, with enough units, can approximate any continuous function on a bounded, closed region of $\mathbb{R}^d$ to any accuracy you ask for (Cybenko, 1989; Hornik, 1991). The theorem does not produce the width, the weights, or a training procedure. The notes use it as a warning against two opposite mistakes: believing "a hidden layer cannot represent XOR" (false, as the next example shows) and believing "a wide sigmoid network will be easy to optimize" (not implied by the theorem).

Example 2.1 (XOR by hand). This is slides 12 and 13 together. The first half is the proof by contradiction that no affine score separates XOR. The second half gives the ReLU network that fits all four points:

$$z^{(1)} = \begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}\begin{pmatrix}x_1\\ x_2\end{pmatrix} + \begin{pmatrix}0\\ -1\end{pmatrix}, \qquad a = \mathrm{ReLU}\big(z^{(1)}\big), \qquad \hat{y} = a_1 - 2a_2.$$

The notes call the second unit an *AND detector*, and describe subtracting twice its output from the "OR-like" first unit as what leaves XOR. (Slide 13's explanation of the first unit as a counter of active inputs is the same idea.) Nine scalar parameters are present if a bias is counted on the output; the output bias here is zero.

Section 3, Definition 3.1 (MLP). Fix layer sizes $n_0 = d$ and $n_1,\dots,n_L$. Set $a^{(0)} = x \in \mathbb{R}^d$. For $\ell = 1,\dots,L$,

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \qquad W^{(\ell)}\in\mathbb{R}^{n_\ell\times n_{\ell-1}}, \quad b^{(\ell)}\in\mathbb{R}^{n_\ell}, \tag{4}$$

$$a^{(\ell)} = f^{(\ell)}\big(z^{(\ell)}\big). \tag{5}$$

Symbols. d is the input size; n_ℓ is the number of neurons in layer ℓ; L is the number of layers (not counting the input); $f^{(\ell)}$ is layer ℓ's activation, applied to each entry separately.

What it says. This is slide 15's recursion, but running all the way to layer L (so the output layer is included). The network's prediction is $a^{(L)}$. In this course the hidden activations $f^{(1)}, \ldots, f^{(L-1)}$ are ReLU unless a problem says otherwise.

Notes page 4: Parameters, losses, and the softmax gradient proof

Parameter counting. The notes repeat slide 16: a map from n_{in} numbers to n_{out} numbers has $n_{\text{out}}n_{\text{in}} + n_{\text{out}}$ parameters, so $784 \to 128 \to 64 \to 10$ has $100{,}480 + 8{,}256 + 650 = 109{,}386$. The first matrix dominates because the input dimension dominates.

Section 4.1, Squared error. For a single predicted number, $L = \frac{1}{2}(\hat{y} - y)^2$ has derivative $\hat{y} - y$ with respect to $\hat{y}$. The $\frac{1}{2}$ only makes that derivative clean; it does not change where the minimum is. For a vector output the same idea is

$$L = \tfrac{1}{2}\|\hat{y} - y\|_2^2, \qquad \nabla_{\hat{y}} L = \hat{y} - y.$$

Symbols. $\|v\|_2^2$ is the sum of the squares of the entries of v. **What it says.** Add up the squared gaps over all outputs and halve; the gradient is the list of gaps.

Section 4.2, Softmax cross-entropy, Definition 4.1. Softmax is equation (6), as on slide 18:

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Subtracting a constant m from every logit multiplies top and bottom by e^{-m} and does not change p. Choosing $m = \max_j z_j$ keeps every exponential between 0 and 1 (slide 19).

With $y \in \mathbb{R}^K$ one-hot (so $y_k \in \{0, 1\}$ and $\sum_k y_k = 1$), the cross-entropy is equation (7):

$$L(z, y) = -\sum_{k=1}^{K} y_k \log p_k = -\log p_{k^\star},$$

where $k^\star$ is the true class. **What it says.** Only the true class's term survives, so the loss is $-\log$ of the probability the network gave to the right answer.

Proposition 4.1 (softmax cross-entropy gradient): $\nabla_z L = p - y$. Slide 17 states this result; the proof lives here. Walk through it in three steps.

Step 1: the notation. Write $S = \sum_j e^{z_j}$ so that $p_i = e^{z_i}/S$. The symbol δ_{ij} here is the *Kronecker delta*: it is 1 when $i = j$ and 0 otherwise. (Do not confuse it with the backprop signal $\delta^{(\ell)}$ from slide 4; the notes reuse the Greek letter for two different things.)

Step 2: how softmax changes. By the quotient rule,

$$\frac{\partial p_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} = p_i(\delta_{ij} - p_j).$$

What it says. $\partial p_i / \partial z_j$ is "how much probability i changes when score j is nudged." The top of the fraction comes from the quotient rule: the derivative of e^{z_i} with respect to z_j is $\delta_{ij} e^{z_i}$ (nonzero only when $i = j$), and the derivative of S with respect to z_j is e^{z_j}. Splitting the fraction gives $p_i\delta_{ij} - p_i p_j$, which is the displayed form.

Step 3: the loss.

$$\frac{\partial L}{\partial z_j} = -\sum_{k=1}^{K} y_k \cdot \frac{1}{p_k} \cdot \frac{\partial p_k}{\partial z_j} = -\sum_{k=1}^{K} y_k(\delta_{kj} - p_j) = -y_j + p_j \sum_{k=1}^{K} y_k = p_j - y_j.$$

What happens. By the chain rule, $\partial L/\partial z_j$ adds up, over every class k, the effect of z_j on p_k times the effect of p_k on the loss, which is $-y_k/p_k$. The factor $1/p_k$ cancels the p_k in $\partial p_k/\partial z_j$. In $-\sum_k y_k \delta_{kj}$ only the term $k = j$ survives, giving $-y_j$. The other piece is $+p_j \sum_k y_k$, and $\sum_k y_k = 1$ because y is one-hot. The result is $p_j - y_j$ for each j, which is $\nabla_z L = p - y$.

Two checks that follow at once. Probabilities sum to 1 by construction. The residual $p - y$ sums to 0 because both p and y sum to 1. If a coded gradient fails either check, the bug is upstream of the parameter update.

Example 4.1 (softmax with exact fractions). Slide 18: $z = (\ln 2, 0, 0)$ gives $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, and for $y = (1, 0, 0)$, $L = \ln 2$ and $\nabla_z L = (-\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, which sums to 0.

Example 4.2 (two classes collapse to a sigmoid). For $K = 2$,

$$p_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2}} = \frac{1}{e^{z_1 - z_2} + 1} = \sigma(z_2 - z_1).$$

What happens. Divide the top and bottom by e^{z_2} to get the middle form, then notice that $1/(1 + e^{-(z_2 - z_1)})$ is the sigmoid of $z_2 - z_1$. If $y = (0, 1)$, Proposition 4.1 says $\partial L/\partial z_2 = p_2 - 1 = \sigma(z_2 - z_1) - 1$, which is Proposition 1.2 applied to the logit difference with label 1. Binary logistic regression and two-class softmax are the same model in two coordinate systems.

Notes page 5: The complete forward pass, computation graphs, and the backprop statement

Section 5, Example 5.1 (both hidden units active). Architecture $2 \to 2 \to 1$, ReLU hidden layer, sigmoid output, binary cross-entropy, one example:

$$x = \begin{pmatrix} 1 \\ 0 \end{pmatrix},\ y = 1,\ W^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix},\ b^{(1)} = \begin{pmatrix} 0 \\ -1/2 \end{pmatrix},\ W^{(2)} = \begin{pmatrix} 1/2 & -1 \end{pmatrix},\ b^{(2)} = 0. \tag{8}$$

This is slides 20 to 22. Layer 1 gives $z^{(1)} = (1, \frac{1}{2})^\top$ and $a^{(1)} = (1, \frac{1}{2})^\top$, so $\mathrm{ReLU}'(z^{(1)}) = (1, 1)^\top$. Layer 2 gives $z^{(2)} = 0$, $\hat{y} = \frac{1}{2}$, $L = \ln 2$.

The cache, explained more carefully. The cache is $(x, z^{(1)}, a^{(1)}, z^{(2)})$. ReLU's backward mask is a function of $z^{(1)}$, not of $a^{(1)}$. If a coordinate of $a^{(1)}$ is zero, the score might have been negative or exactly zero; the activation value does not say which, and those two cases are exactly the boundary of the subgradient. So store z.

Section 6, Computation graphs. The multivariate chain rule, for $L = L(r)$ with $r = r(u, y)$ and $u = u(w, x)$, is ordinary multiplication along a path; if two paths reach the same variable, their contributions add. This is slide 24 in words.

Example 6.1 (scalar graph). Slide 23: $w = 2$, $x = 3$, $y = 4$, $u = wx$, $r = u - y$, $L = \frac{1}{2}r^2$ give $u = 6$, $r = 2$, $L = 2$, with local derivatives $\partial u/\partial w = 3$, $\partial u/\partial x = 2$, $\partial r/\partial u = 1$, $\partial L/\partial r = 2$. So

$$\frac{\partial L}{\partial w} = 2 \cdot 1 \cdot 3 = 6, \qquad \frac{\partial L}{\partial x} = 2 \cdot 1 \cdot 2 = 4.$$

The closed form $L = \frac{1}{2}(wx - y)^2$ gives the same $\partial L/\partial w = (wx - y)x = 6$. The notes add: backpropagation is this arithmetic, organized from the loss backward so each in-between derivative is computed only once.

Proposition 6.1 (one dense layer). Let $z = Wx + b$ with $W \in \mathbb{R}^{m \times d}$, and let $\delta = \nabla_z L$ be known. Then

$$\nabla_W L = \delta\, x^\top, \qquad \nabla_b L = \delta, \qquad \nabla_x L = W^\top \delta. \tag{9}$$

Proof. The coordinate form $z_i = \sum_{j=1}^{d} W_{ij} x_j + b_i$ gives

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z_i} x_j = \delta_i x_j, \qquad \frac{\partial L}{\partial b_i} = \delta_i, \qquad \frac{\partial L}{\partial x_j} = \sum_{i=1}^{m} W_{ij} \delta_i,$$

which are the three displayed products. This is slide 25; see it for the symbol-by-symbol explanation. The notes call equation (9) the entire fully connected backward step: a nonlinearity between $z^{(\ell)}$ and $a^{(\ell)}$ just inserts one Hadamard product with $f'(z^{(\ell)})$ before (9) is applied.

Section 7, Proposition 7.1 (two-layer binary MLP). Consider $a^{(1)} = f(z^{(1)})$, $z^{(1)} = W^{(1)}x + b^{(1)}$, $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$, $\hat{y} = \sigma(z^{(2)})$, and binary cross-entropy. Then

$$\delta^{(2)} = \hat{y} - y, \tag{10}$$

$$\nabla_{W^{(2)}} L = \delta^{(2)} \big(a^{(1)}\big)^\top, \qquad \nabla_{b^{(2)}} L = \delta^{(2)}, \tag{11}$$

$$\delta^{(1)} = \Big(\big(W^{(2)}\big)^\top \delta^{(2)}\Big) \odot f'\big(z^{(1)}\big), \tag{12}$$

$$\nabla_{W^{(1)}} L = \delta^{(1)} x^\top, \qquad \nabla_{b^{(1)}} L = \delta^{(1)}. \tag{13}$$

What it says. These are slide 26's formulas written for two layers: start with the output error (10), get the output layer's gradients (11), send the error back through the weights and the activation slope (12), and get the first layer's gradients (13). The proof continues on page 6.

Notes page 6: Proof of backprop, the full one-step arithmetic, and the batch layout

The L-layer version. For a network with L layers the hidden step (12) simply repeats:

$$\delta^{(\ell)} = \Big(\big(W^{(\ell+1)}\big)^\top \delta^{(\ell+1)}\Big) \odot f'\big(z^{(\ell)}\big).$$

Proof of Proposition 7.1, in plain steps.

1. Equation (10) is Proposition 1.2.
2. Equations (11) are Proposition 6.1 applied at layer 2, where the incoming activation is $a^{(1)}$.
3. The same proposition applied to $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$ also gives $\nabla_{a^{(1)}} L = (W^{(2)})^\top \delta^{(2)}$: that is how the error reaches the hidden outputs.
4. Going from the hidden outputs to the hidden scores, the entry-by-entry chain rule through $a^{(1)} = f(z^{(1)})$ multiplies by $f'(z^{(1)})$. That gives (12).
5. Equations (13) are Proposition 6.1 applied at layer 1, where the incoming signal is x.

Mean loss versus sum. If the training objective is the *mean* of n per-example losses, every gradient in Proposition 7.1 is replaced by the average of the per-example gradients. Equivalently, add up the δ contributions and divide by n. Mixing a mean loss with a sum of gradients rescales the learning rate by n (compare slide 17).

Example 7.1 (backward pass for the running example). This is slides 27 and 28:

$$\delta^{(2)} = -\tfrac{1}{2}, \quad \nabla_{W^{(2)}} L = \begin{pmatrix} -1/2 & -1/4 \end{pmatrix}, \quad \nabla_{b^{(2)}} L = -\tfrac{1}{2}, \quad \delta^{(1)} = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix},$$

$$\nabla_{W^{(1)}} L = \begin{pmatrix} -1/4 & 0 \\ 1/2 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}.$$

The notes repeat the key point: the second column of $\nabla_{W^{(1)}} L$ is zero because $x_2 = 0$, not because of ReLU. Both hidden units are active.

Example 7.2 (one gradient step), all the arithmetic. With $\eta = \frac{1}{10}$ and the rule new = old $- \eta \cdot$

gradient (slide 29 shows the results; here are the steps):

$$
\begin{aligned}
W^{(2)} &\leftarrow \begin{pmatrix}1/2 & -1\end{pmatrix} - \tfrac{1}{10}\begin{pmatrix}-1/2 & -1/4\end{pmatrix} = \begin{pmatrix}11/20 & -39/40\end{pmatrix},\\
b^{(2)} &\leftarrow 0 - \tfrac{1}{10}\Big(-\tfrac{1}{2}\Big) = \tfrac{1}{20},\\
W^{(1)} &\leftarrow \begin{pmatrix}1 & -1\\ 1 & 1\end{pmatrix} - \tfrac{1}{10}\begin{pmatrix}-1/4 & 0\\ 1/2 & 0\end{pmatrix} = \begin{pmatrix}41/40 & -1\\ 19/20 & 1\end{pmatrix},\\
b^{(1)} &\leftarrow \begin{pmatrix}0\\ -1/2\end{pmatrix} - \tfrac{1}{10}\begin{pmatrix}-1/4\\ 1/2\end{pmatrix} = \begin{pmatrix}1/40\\ -11/20\end{pmatrix}.
\end{aligned}
$$

Recompute the forward map on the same input $x = (1, 0)^\top$:

$$
z^{(1)} = \begin{pmatrix}41/40\\ 19/20\end{pmatrix} + \begin{pmatrix}1/40\\ -11/20\end{pmatrix} = \begin{pmatrix}21/20\\ 2/5\end{pmatrix}, \qquad a^{(1)} = z^{(1)} \text{ (both entries stay positive)},
$$

$$
z^{(2)} = \tfrac{11}{20}\cdot\tfrac{21}{20} + \Big(-\tfrac{39}{40}\Big)\cdot\tfrac{2}{5} + \tfrac{1}{20} = \frac{231 - 156 + 20}{400} = \frac{95}{400} = \frac{19}{80}.
$$

Reading it. The first term is $\frac{231}{400}$. The second is $-\frac{78}{200} = -\frac{156}{400}$. The third is $\frac{20}{400}$. They add to $\frac{95}{400} = \frac{19}{80}$. Now $\hat{y} = \sigma(19/80) = \sigma(0.2375) \approx 0.559$ and $L = -\log \hat{y} \approx 0.581$, down from $\ln 2 \approx 0.693$. The step moved the prediction toward the label. A full training run repeats this on a batch for many steps. Choosing η, averaging noise, and momentum are Week 2.

Section 8, Batch shapes and the NumPy layout (start). The home practice stores a batch as $X \in \mathbb{R}^{n\times d}$, one example per row, and uses $Z^{(1)} = XW^{(1)}_{\text{code}} + b^{(1)}$ (continued on page 7).

Notes page 7: The NumPy layout, the autograd check, and the eight problems

Layout, continued. $W^{(1)}_{\text{code}} \in \mathbb{R}^{d\times h}$ is the transpose of the board matrix $W^{(1)} \in \mathbb{R}^{h\times d}$, where h is the number of hidden units. The bias is added to every one of the n rows (broadcast).

The batch backward pass for a mean binary loss. The home practice sets

$$
dZ^{(2)} = \frac{\hat{Y} - Y}{n} \in \mathbb{R}^{n\times 1},
$$

and then

$$
\begin{aligned}
dW^{(2)}_{\text{code}} &= \big(A^{(1)}\big)^\top dZ^{(2)}, \qquad db^{(2)} = \sum_{\text{rows}} dZ^{(2)},\\
dA^{(1)} &= dZ^{(2)}\big(W^{(2)}_{\text{code}}\big)^\top, \quad dZ^{(1)} = dA^{(1)} \odot \mathbf{1}[Z^{(1)} > 0],\\
dW^{(1)}_{\text{code}} &= X^\top dZ^{(1)}, \qquad db^{(1)} = \sum_{\text{rows}} dZ^{(1)}.
\end{aligned}
$$

Symbols. Same as slide 30. $\sum_{\text{rows}}$ means "add up all n rows, leaving one row." **What it says.** The weight gradients are transposed activations times the stored error signals. The bias gradient is the column-wise total of the error signals, because the bias was added to every row, so its effect on the loss is the sum over rows. The ReLU mask uses $Z^{(1)}$.

Check on the running example. With $n = 1$, the board row $\nabla_{W^{(2)}} L = (-\frac{1}{2}, -\frac{1}{4})$ and the code column $dW^{(2)}_{\text{code}} = (-\frac{1}{2}, -\frac{1}{4})^\top$ are transposes of each other. They describe one linear map.

Autograd check. Copy the NumPy weights into PyTorch parameters, run the same forward expression, call the backward routine on the loss, and compare. A relative error below 10^{-5} on every parameter is the acceptance test in the home practice. The usual failures are: a missing $1/n$, a bias summed over the wrong axis, or a ReLU mask built from A instead of Z.

Section 9, The problems (what each one asks).

- **P1.** Compute $\sigma(0)$, $\sigma'(0)$, and $\tanh'(0)$. Prove $\sigma'(z) \le \frac{1}{4}$.

- **P2.** Prove Proposition 1.1 (the sigmoid derivative) from the definition, without looking it up mid-proof, then prove Proposition 1.2 (the BCE gradient).
- **P3.** Finish the XOR contradiction in your own lines, then evaluate the ReLU XOR network at $(1,1)$ and at $(1,0)$, showing each pre-activation.
- **P4.** Count the parameters of a $4 \to 3 \to 2$ network, write every matrix shape in the board convention, and for a batch of 32 in the code convention write the shapes of X, $W^{(1)}_{\text{code}}$, $Z^{(1)}$, and $dW^{(1)}_{\text{code}}$.
- **P5.** With $z = (0, \ln 3)$ and $y = (0,1)$, compute the softmax, the cross-entropy, and $\nabla_z L$; verify $\partial L/\partial z_2 = \sigma(z_2 - z_1) - 1$.
- **P6 (main calculation).** The dead-unit network from slide 35: forward pass, every parameter gradient, identify the dead unit, and list the gradient entries it forces to zero.
- **P7.** Using the gradients of Example 7.1 and $\eta = \frac{1}{10}$, recompute only $W^{(2)}$ and $b^{(2)}$ (leave layer 1 alone). Evaluate the new $z^{(2)}$ with the original $a^{(1)} = (1, \frac{1}{2})^\top$ and show the new $\hat{y}$ is larger than $\frac{1}{2}$.
- **P8.** In four to six sentences, explain why the ReLU backward pass must consult $z^{(1)}$, then say what happens to $\nabla_{W^{(1)}} L$ on an example where every hidden unit is dead.

Notes page 8: Solutions P1 to P6

P1. $\sigma(0) = \frac{1}{2}$, $\sigma'(0) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$, and $\tanh'(0) = 1 - \tanh^2(0) = 1$ (because $\tanh 0 = 0$). The bound $\sigma'(z) \le \frac{1}{4}$ is the second half of Proposition 1.1: $s(1-s) \le \frac{1}{4}$ for s in $(0,1)$. See slide 40 for the explanation.

P2. The proofs are Proposition 1.1 (slide 11) and Proposition 1.2 (slide 9). The notes' grading hint: a solution that starts from $\partial L/\partial \hat{y}$ and multiplies by σ' is complete; a solution that only quotes $\hat{y} - y$ is not.

P3. The contradiction is Example 2.1 (slide 12). At $(1,1)$:

$$z^{(1)} = (2,1)^\top, \qquad a = (2,1)^\top, \qquad \hat{y} = 2 - 2 \cdot 1 = 0.$$

At $(1,0)$:

$$z^{(1)} = (1,0)^\top, \qquad a = (1,0)^\top, \qquad \hat{y} = 1.$$

Why. At $(1,1)$ the scores are $z_1 = 1 + 1 = 2$ and $z_2 = 1 + 1 - 1 = 1$, both positive, so ReLU leaves them alone and $\hat{y} = a_1 - 2a_2 = 2 - 2 = 0$ (the XOR label for $(1,1)$). At $(1,0)$ the scores are $z_1 = 1$ and $z_2 = 1 + 0 - 1 = 0$; the second is exactly zero, so ReLU gives 0 and $\hat{y} = 1 - 0 = 1$ (the XOR label).

P4. Same as slide 41: $W^{(1)} \in \mathbb{R}^{3\times 4}$, $b^{(1)} \in \mathbb{R}^3$, $W^{(2)} \in \mathbb{R}^{2\times 3}$, $b^{(2)} \in \mathbb{R}^2$, and $(3 \cdot 4 + 3) + (2 \cdot 3 + 2) = 15 + 8 = 23$. For $n = 32$: X is 32×4, $W^{(1)}_{\text{code}}$ is 4×3, $Z^{(1)}$ is 32×3, and $dW^{(1)}_{\text{code}}$ has the same shape as $W^{(1)}_{\text{code}}$, namely 4×3. (A gradient always has the same shape as the thing it is the gradient of.)

P5. Same as slide 44. $e^z = (1,3)$ and the normalizer is 4, so $p = (\frac{1}{4}, \frac{3}{4})$. The true class is class 2, so $L = -\log \frac{3}{4} = \log \frac{4}{3}$ and $\nabla_z L = (\frac{1}{4}, -\frac{1}{4})$. The logit difference is $z_2 - z_1 = \ln 3$, and $\sigma(\ln 3) = \dfrac{1}{1 + 1/3} = \frac{3}{4}$, so $\sigma(\ln 3) - 1 = -\frac{1}{4} = \partial L/\partial z_2$.

P6. Same as slides 42 and 43. Forward:

$$z^{(1)} = \begin{pmatrix} 1 - 1/2 \\ 1 - 3/2 \end{pmatrix} = \begin{pmatrix} 1/2 \\ -1/2 \end{pmatrix}, \qquad a^{(1)} = \begin{pmatrix} 1/2 \\ 0 \end{pmatrix},$$

$$z^{(2)} = 2 \cdot \tfrac{1}{2} + 2 \cdot 0 - 1 = 0, \qquad \hat{y} = \tfrac{1}{2}, \qquad L = -\log \tfrac{1}{2} = \ln 2 \ \text{ (because } y = 0).$$

(The printed notes write the second entry of $z^{(1)}$ as -1; the correct value is $-\frac{1}{2}$, as explained on slide 42. Nothing after it changes.) Backward:

$$\delta^{(2)} = \tfrac{1}{2} - 0 = \tfrac{1}{2}, \qquad \nabla_{W^{(2)}} L = \tfrac{1}{2} \begin{pmatrix} 1/2 & 0 \end{pmatrix} = \begin{pmatrix} 1/4 & 0 \end{pmatrix}, \qquad \nabla_{b^{(2)}} L = \tfrac{1}{2},$$

$$\delta^{(1)} = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \cdot \tfrac{1}{2} \odot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \nabla_{W^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The dead unit. Unit 2 is dead because its score is negative. That zero in the ReLU mask forces the second row of $\nabla_{W^{(1)}} L$ and the second entry of $\nabla_{b^{(1)}} L$ to vanish. The outgoing weight is also zero in the gradient, for a second reason: $a_2^{(1)} = 0$, so the second entry of $\nabla_{W^{(2)}} L = \delta^{(2)} (a^{(1)})^\top$ vanishes even before thinking about the mask. On this example the dead unit is frozen: its incoming weights, bias, and outgoing weight do not move.

Notes page 9: Solutions P7 and P8, and the footer

P7. Keep $a^{(1)} = (1, \frac{1}{2})^\top$ as in Example 5.1. From Example 7.2 the updated output layer is $W^{(2)} = (11/20,\ -39/40)$ and $b^{(2)} = 1/20$. Then

$$z^{(2)} = \tfrac{11}{20} \cdot 1 + \Big(-\tfrac{39}{40} \Big) \cdot \tfrac{1}{2} + \tfrac{1}{20} = \tfrac{22}{40} - \tfrac{39}{80} + \tfrac{4}{80} = \frac{44 - 39 + 4}{80} = \frac{9}{80} > 0.$$

Step by step. Write everything over 80: $\frac{11}{20} = \frac{44}{80}$, $\frac{39}{80}$ stays, and $\frac{1}{20} = \frac{4}{80}$. Then $44 - 39 + 4 = 9$. Since $z^{(2)} = \frac{9}{80} > 0$, we get $\hat{y} = \sigma(9/80) > \sigma(0) = \frac{1}{2}$. Updating only the output layer already moves the prediction toward $y = 1$. Updating layer 1 as well, as Example 7.2 does, moves it further, to $\sigma(19/80)$.

P8. The expected answer, in plain words.

- ReLU's slope at a coordinate is 1 when the score is positive and 0 when it is negative.
- The activation stores only the positive part, so an activation of 0 fits both a negative score and a zero score. The activation therefore cannot tell you the slope.
- The mask has to be computed from the stored score $z^{(1)}$.
- If every hidden unit is dead on an example, then $f'(z^{(1)}) = 0$, hence $\delta^{(1)} = 0$, hence $\nabla_{W^{(1)}} L = \delta^{(1)} x^\top = 0$ and $\nabla_{b^{(1)}} L = 0$ on that example. The example teaches the first layer nothing.
- If this happens for every example in the batch, the first-layer update is the zero matrix.

The footer line at the bottom of the page names the lecture notes, the companion slides file, and the home practice file.

Common mistakes the course warns about

These are collected from across the slides and notes, because Quiz 1 can ask for a verbal reason.

- **Mean versus sum.** Using a mean loss with a sum-style gradient (or the reverse) changes the learning rate by a factor of n (slides 17, 31; notes pages 6, 7).
- **ReLU mask from the wrong array.** Build it from Z, not A (slides 21, 22, 31; notes pages 5, 7, 9).
- **Mixing up the two zeros.** A zero gradient column means a zero input (slide 28); a zero gradient row means a dead unit (slide 43).
- **Bias shapes and axes.** $(h,)$ versus $(1, h)$, and summing the bias gradient over rows rather than columns (slide 31; notes page 7).
- **Output error that does not sum to zero** in the softmax case: a bug upstream of the update (slide 18; notes page 4).
- **Forming σ then** log for large scores, and exponentiating without subtracting the max: use the stable forms (slide 19; notes page 2).
- **Squared error on a sigmoid output.** It leaves an extra σ' that kills the gradient when the network is confidently wrong (slide 9).

- **Confusing a hidden layer's expressive power with easy training.** The universal approximation theorem says a good network exists, not that it is easy to find (slide 39; notes page 3).

Small inconsistencies noticed in the source

- **Arithmetic slip in Problem 3.** Slide 42 and notes page 8 give the second entry of $z^{(1)}$ as -1; the correct value is $-\frac{1}{2}$. The conclusions (unit 2 dead, all gradients) are unchanged.
- **Unresolved cross-references.** The printed lecture notes show "??" wherever they refer to a numbered section, equation, or proposition, and the contents list is empty. The explanations above match each reference by content.
- **Two names for one thing.** The slides say "Home Assignment"; the notes say "Home Practice."
- **Two meanings of δ.** $\delta^{(\ell)}$ is the backprop signal; δ_{ij} in the softmax proof is the Kronecker delta (notes page 4).
- **Layer index range.** Slide 15 runs the layer rule to $L-1$ and treats the output layer separately; the notes' Definition 3.1 runs it to L.
- **Naming the network output.** Slide 13 calls the XOR network's output y; the notes call it $\hat{y}$. It is the network's output, not the label.